

Modeling and Control of a PMSM Operating in Low Speeds

Robin Helsing & Tobias Sanchez

Master of Science Thesis in Electrical Engineering
Modeling and Control of a PMSM Operating in Low Speeds

Robin Helsing & Tobias Sanchez
LiTH-ISY-EX-22/5504-SE

Supervisor: **PhD Student Arvind Balachandran**
ISY, Linköpings universitet
Magnus
The corporation

Examiner: **Mattias Krysanter**
Associate Professor
ISY/IDA, Linköpings universitet

*Division of Automatic Control
Department of Electrical Engineering
Linköping University
SE-581 83 Linköping, Sweden*

Copyright © 2022 Robin Helsing & Tobias Sanchez

Sammanfattning

En synkronmotor med permanentmagnet är en motortyp som används inom flera olika applikationsområden, inte minst i en autonom robot där denna motortyp kan driva hjulen. Idag väljer många företag simulering som ett verktyg för att spara pengar och tid vid testning av produkt. Den här avhandlingen täcker både processen att modellera en synkronmotor med permanentmagneter samt att reglera den vid låga hastigheter, i en simuleringsmiljö.

Motorn är som tidigare nämnts en synkronmotor med permanentmagnet och är en direktdriven outrunner, vilket innebär att motorn och hjulet är sammanslagna och att rotern sitter utanför statorn. På nuvarande robot i produktion sitter en utväxling mellan motor och hjul för att kunna reglera motorn på högre varv och således generera ett moment. Utväxlingen bidrar till förluster samt är en extra kostnad, därför är undersökningen av en direktdriven motor intressant.

Den direktdrivna motorn har en lägre arbetshastighet och är av vissa skäl därför svårare att reglera vid pålastning av motorn. Motorn är försedd med strömsensorer och en positionssensor, vilken har en viss upplösning. Positionssensorn är hastighetsberoende i den mening att vid lägre varv erhålls färre mätningar, vilket är ett problem vid regleringen av motorn.

Avhandlingen undersöker två olika reglerstrategier, en är en mer klassisk PI-reglering som ofta används ute på marknaden i diverse system och den andra är model predictive control (MPC). Den sistnämnda är en online optimering där, med hjälp av information om systemet, en optimal insignal räknas ut och appliceras.

Två olika Kalman filter implementeras tillsammans med de två olika reglerstrategierna, för att estimerast hastigheten med hjälp av mätningarna från ström- och positionssensor.

Slutsatsen är en ideal motormodell som efterliknar den fysiska motorn. MPC klarar att reglera motorn mellan 0-50 RPM, både med och utan pålagt moment. Än bättre med hastighetsestimering från ett Kalman filter. PI-regulatorn klarar inte av att reglera motorn vid 2 RPM men vid 10 RPM och uppåt klarar den det, dock resulterar den i över- och undersläng vid accelerationer.

Abstract

A permanent magnet synchronous motor is a type of motor that is used in several different application areas, not least in an autonomous robots where it is the motor that drives the wheels. Today, many actors choose simulation as a tool to save money and time when product tests are performed. This thesis covers both the process of modeling a permanent magnet synchronous motor and regulating it at low speeds, in a simulation environment.

As previously mentioned, the motor is a permanent magnet synchronous motor and is a direct-driven outrunner, which means that the motor and the wheel are combined and that the rotor is spinning outside the stator. On current robots in production, there is a gear ratio between the motor and wheels to be able to regulate the motor at higher speeds and thus generate a torque. The gearing contributes to losses and is an extra cost, so the examination of a direct-drive motor is interesting.

The direct-drive motor has a lower working speed and is therefore by some reasons more difficult to regulate when applying torque load to the motor. The motor is equipped with current sensors and a position sensor, which has a certain resolution. The position sensor is speed-dependent in the sense that at lower RPMs fewer measurements are obtained, which is a problem when regulating the motor.

The thesis examines two different control strategies, one of which is a more classic PI control that is often used on the market in various systems and the other is model predictive control (MPC). The latter is an online optimization where, with the help of information about the system, an optimal input signal is calculated and applied.

Two different non-linear Kalman filters are also examined, which are implemented with the two different control strategies, to estimate the speed with the help of the measurements from current and the position sensor.

The conclusion is an ideal motor model that mimics the physical motor. MPC is able to regulate the motor between 0-50 RPM, both with and without applied torque and even better with speed estimation from a Kalman filter. The PI controller is not able to regulate the motor at 2 RPM but for speeds at 10 RPM and greater, however with over-/undershoot after an acceleration.

Acknowledgments

There are a few people that has meant a lot during this project, which we would like to acknowledge. This semester has been both enjoyable in such way that it went smoothly but also challenging when things did not went as anticipated. In the end, it has been a fun project where we have learned a lot. However, we would never have made it without the help we received from our supervisors at the university and at the corporation.

First we would like to thank our supervisor at the company, Magnus. You have been a big source of information and has almost always been able to answer our technical questions when it comes to the motor. You have also introduced us to the *real world*, in terms of how the job as an engineer is in real life. Then we have Arvind Balachandran, our supervisor at ISY. You have been a real lifesaver when it comes to the theoretical questions, both regarding the motor but control theory as well. You have instantaneously answered them, you have provided us with great literature and you have been a great person to bounce ideas with during this period, and not least a good friend.

Linköping, June 2022
Robin Helsing and Tobias Sanchez

Contents

Notation	xi
1 Introduction	1
1.1 Problem Formulation	2
1.1.1 Motivation	2
1.1.2 Purpose	3
1.1.3 Problem statement	3
1.1.4 Delimitations	4
1.2 Related work	4
1.3 Individual contributions	5
2 Permanent Magnet Synchronous Motor	7
2.1 Working principle	7
2.1.1 Inverter	9
2.2 Field Oriented Control	10
2.2.1 Clarke	11
2.2.2 Park	12
2.3 d-q modelling	14
3 Open loop	15
3.1 Model	16
3.2 Parameters	17
3.3 Validation	18
4 Filtering and Sensoring	23
4.1 Position sensor	23
4.2 Filter theory	26
4.2.1 Extended Kalman Filter	27
4.2.2 Unscented Kalman Filter	29
5 Controlling	33
5.1 Control theory	34
5.1.1 PI	34

5.1.2	Model Predictive Control	39
5.2	Implementation	43
5.2.1	PI implementation	43
5.2.2	Model Predictive Control implementation	44
6	Results	47
6.1	Test 1 - Max-min speed & max torque	48
6.2	Test 2 - Smooth start and stop	52
6.3	Test 3 - Speed torque	54
6.4	Test 4 - Worst case	56
7	Discussion	59
7.1	Modelling	59
7.2	Controlling	61
8	Conclusions	63
8.1	Conclusions	63
8.2	Future work	64
A	Appendix	67
A.1	Motor requirements	67
A.2	Controller parameters	68
A.3	Simulink models	69
A.4	EKF/UKF algorithm	78
	Bibliography	83

Notation

NOMENCLATURE

Notation	Meaning
s	Laplace-transform variable
d	Direct-axis
q	Quadrature-axis
dq	direct-quadrature
α	α -axis
β	β -axis
ω_m	Mechanical speed
ω_e	Electrical speed
θ	Rotor angle
Ψ_R	Magnetic flux linkage
n_p	Pole pairs
u	Phase U
v	Phase V
w	Phase W
R_s	Phase resistance
L_d	Direct-axis inductance
L_q	Quadrature-axis inductance
T_e	Electrical torque
T_L	Load torque
b	Viscous damping constant
J	Inertia
ref	Reference
ppr	Points per revolution

OPERATOR

Operator	Meaning
$\hat{x}_{i j}$	Estimated state x in time step i given the information from the step j .
x_{k-1}	Discrete state x in time step $k-1$.
$\hat{x}_k^i$	i :th sigma point for time step k .

ABBREVIATIONS

Abbreviation	Meaning
PMSM	Permanent magnet synchronous motor
PI	Proportional, integral (regulator)
MPC	Model predictive control
EKF	Extended Kalman filter
UKF	Unscented Kalman filter
UT	Unscented transform
FOC	Field oriented control
PWM	Pulse width modulation
RMS	Root mean square
AC	Alternating current
DC	Direct current
SPWM	Sinusoidal pulse width modulation
OCP	Optimal control problem

1

Introduction

This thesis was done during the spring term in 2022. It was done at Linköping University at ISY as a collaboration with a robot corporation¹. The corporation develops and manufactures autonomous robots amongst other, which is the product investigated in this thesis. The corporation's R&D department are developing a new driveline on the robot, that includes a new electrical hub motor that is to replace the old version. The old version had a gearing so that the motor could run at higher speeds but the gearing also introduces transmission and efficiency losses and therefore the hub solution is being investigated. This motor and the controller of it is the focus of this thesis. The motor is modelled in Simulink [4] and validated with the physical motor and thereafter two control strategies were investigated together with two different Kalman filters, to enhance the speed estimate at lower speeds in particular.

¹ As the thesis is a collaboration with a robot corporation whereof the motor is not on the market and the development is at an early stage the company will be referred as *the corporation*.

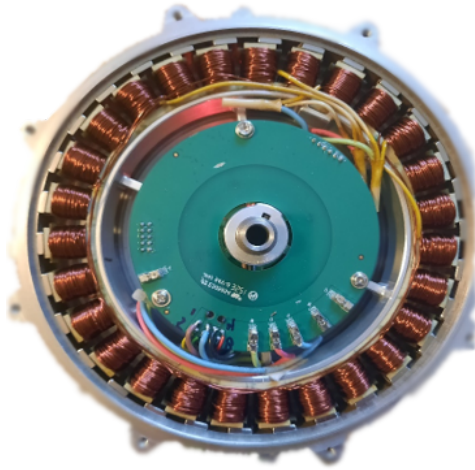


Figure 1.1: The hub motor investigated in this thesis.

1.1 Problem Formulation

The problem formulation includes *Motivation* that describes what will be investigated, how and why, *Purpose* describes the background and history as well as why and from where the need for this investigation comes from. *Problem statement* states the three points of investigation in this thesis and the *Delimitations* states parts that will not be considered in the thesis. *Related work* describes other research on the same topic that could be of interest and *Individual contributions* describes what part of the thesis each of us writers have mainly contributed to.

1.1.1 Motivation

This thesis will investigate different methods of controlling a direct driven permanent magnet synchronous motor (PMSM) on an autonomous robot, see figure 1.1. The former system had a gearing between the motor and the wheel in order to operate the motor at a more efficient operating point to achieve high torque and to maintain smooth control at low speeds in different terrains. The motor's operational region is 0-50 RPM and a torque load at 0-5 Nm with a maximum load at 8 Nm.

Simulation of dynamical systems is a manner of exploring if the systems performs as wanted, and it is cheaper and safer compared to physical testing. Therefore, the goal is to first model the motor in Simulink and then to control it for its different operation points such that the robot's performance is agile. Two different control strategies are investigated. One is a conventional Proportional Integral-

controller (PI) that is widely used for all kinds of different control systems. The other strategy investigated is model predictive control (MPC), which is a more advanced strategy in terms of user-friendliness and tuning parameters but it generates an optimized control signal. The controller should be agile, which means, for example, that for one scenario smooth start and stops but in another scenario a fast response might be wanted. It should be able to control the motor so that it can maintain a certain torque within its operating speed.

The speed sensing in the motor setup is speed-dependable, meaning that the measurement availability is less frequent for lower speeds. Two different filters are to be implemented in order to estimate the motor speed and to get a more accurate and consistent measurement.

1.1.2 Purpose

The industry has made great strides since the beginning of its era. In the beginning of the 20th century the first manually operated "robot" was introduced and almost 90 years later the autonomous robot was created. One of the goals today is to optimize and streamline this type of robot in terms of updating the software and the control algorithms to lower production cost and extend lifetime and lower owner cost. The autonomous robot has been produced by the corporation for more than 25 years and this thesis is a part of the future development of its motor. As the gearing is a matter of cost, weight and complexity the corporation is scouting for new solutions and is currently investigating a direct drive hub motor, i.e. removing the gearing. When doing so the design of the control system is in need of an update in order to achieve the wanted behaviour. Using the built in position sensor together with current sensors the rotor position and current feedback can be used to control the commutation and establish an efficient and adaptive control loop. As the corporation is looking for a way of evaluating different control strategies in a simulation environment the motor model resemblance with the physical one is of importance. The purpose of this thesis is therefore to build a simulation model of the motor and to investigate different control strategies, including estimation of the mechanical speed.

1.1.3 Problem statement

The problems that are to be answered in this thesis are

- How accurate is the simulated motor model compared to the physical motor in terms performance²?
- How does a MPC and a PI controller perform applied to the motor at its operational region?
- Does an Extended Kalman Filter/Unscented Kalman Filter estimate the lower speeds closer to the true speed than the estimate from the position sensor?

²Performance meaning the efficiency, RMS currents and voltages and speed-torque characteristics.

1.1.4 Delimitations

The delimitations made in order to finish the project on time and to be able to deliver a final result to the corporation are listed as follows:

- The battery in the robot is assumed to be an ideal voltage source, i.e. it can deliver the needed power to the motor. No current limitations considered.
- The inverter is not taken in to account and is therefore not modelled nor taken in to consideration. See Section 2.1.1.
- The autonomous robot has two driving wheels, but the controlling and modelling will only consider one.
- Identification of optimal motor parameters will not be made, but only tuned to fit test-data from the physical motor.

1.2 Related work

There are plenty of research articles within the subject of controlling a PMSM in several databases, but there are however none or very few that deals with the topic of controlling a motor in such low speeds as the motor in this thesis. It seems to be a narrow and niche area. In [9] the motor is controlled with a PI-regulator for both the inner current loops and the outer speed loop, whereof the outer is a fuzzy controller meaning that the regulator is tuned online, from a table based on expert experience. The result is an improved controller with less overshoot compared to a normal PI-regulator, but one has to keep in mind that the operating RPM is 20x the motor investigated in this thesis, meaning that the speed and acceleration measurement resolution is greater.

In [7], which was inspirational to the model predictive control strategy, a load torque observer is implemented to predict eventual load torque applied to the motor. The result is a controller that takes the disturbances into account and the impact on the speed is close to zero. But as well as in the papers previously mentioned the speed is a lot greater.

1.3 Individual contributions

As this thesis is written and accomplished by two students the work has been performed both individually and jointly. The practical part has been divided such that Robin investigated the MPC and the EKF, while Tobias did the UKF and the PI-controller. The modelling of the motor has been done together but the position sensor Tobias contributed to mostly. The report is mostly written together besides the individual practical areas, which were written by the person investigating them for practical reasons. There are also other parts in the report that were written individually, such as Robin writing the PI-tuning and Open loop sections while Tobias wrote the position sensor, inverter and FOC. Overall the work has been divided equally and fair, meaning both contributed as much as expected. Results and conclusions have been written very much jointly.

2

Permanent Magnet Synchronous Motor

This chapter describes the basic working principle of a Permanent Magnet Synchronous Motor (PMSM) that is the basis of this work, the connection-type of the motor and uncovers the concept of Field Oriented Control (FOC) with its Clarke & Park transformations and the mathematical models of the PMSM in the dq-frame. The inverter in the system is briefly explained as well and an explanation is given to why it is negligible. The dq-frame is the final mathematical model which is used to generate the Simulink model of the motor.

2.1 Working principle

A PMSM is a brushless electric motor that is rotating because of an induced magnetic field. The motor has two main parts, a rotor and a stator. The stator is the non-moving part and is holding the electromagnets. The rotor is the part that holds the permanent magnets and is rotating. On an outrunner, which is the type being investigated, the rotor is rotating outside the stator. The working principle of the motor is such that a current is sent through an electromagnetic coil. As the coil is magnetized a permanent magnet, with opposite polarity, on the rotor will try to align with the coil and a magnet with the same polarity will repulse from the electromagnet. Further, another coil is powered, and the rotor keeps turning, and on it goes. This is illustrated in Figure 2.1. Sending a three-phase current with some frequency, the motor will rotate with the same speed as the frequency divided by the number of pole pairs. A more detailed explanation of a PMSM can be found in [12] amongst other.

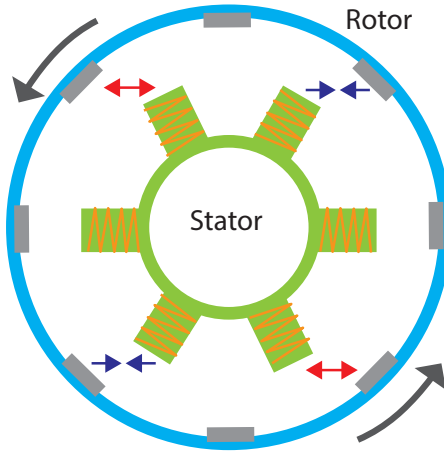


Figure 2.1: Working principle of a PMSM.

The motor designed by the corporation is a Δ -connected PMSM motor. In a Δ -connection the phases are connected individually as seen in Figure 2.2. Δ connection is not the most popular connection, compared to Y-connected one, due to the fact that it has a major weakness. That being the ohmic motor losses, I^2R . Because of this the Δ -connected motors mostly appears in motors with low power output where the losses can be outweighed with lower material costs [8].

The connection of the motor does not affect the dq-modelling in any way but are resulting in losses.

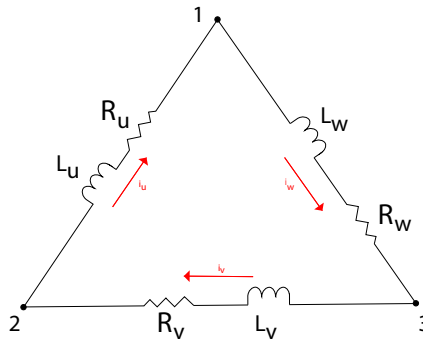


Figure 2.2: Delta connection

The figure 2.2 shows a Δ -connected PMSM where the nodes 1, 2 and 3 are the connection points and the L & R represents the resistance and inductance of each phase.

2.1.1 Inverter

An inverter is an electronic device that transforms direct current (DC) to alternating current (AC). The three phase voltage and current to the motor is generated with sinusoidal pulse width modulation (SPWM), which converts the DC power to AC. The input of the SPWM is the signal $i_{uvw,ref}(t)$ and the V_{DC} from the battery and the output is a three phase current/voltage $i_{uvw}(t)/V_{uvw}(t)$.

A triangle wave carrier signal, $s(t)$, is generated with a frequency f_s and the reference signal (i_{ref} from the controller) is compared with the carrier signal and a PWM signal is generated as following.

$$\text{PWM}(t) = \begin{cases} V_{DC}, & \text{for } s(t) \geq i_{uvw,ref}(t), \\ -V_{DC}, & \text{for } s(t) < i_{uvw,ref}(t). \end{cases} \quad (2.1)$$

The PWM is controlling the amount of voltage that is to be sent to the motor. Equation (2.1) is plotted in Figure 2.3. Since the frequency of the carrier signal and the frequency of the control system is the same, there is no loss in the PWM resolution. Furthermore, the PWM frequency is at a level so that the output from the inverter can be assumed to be an ideal three-phase signal, therefore the model does not include the inverter.

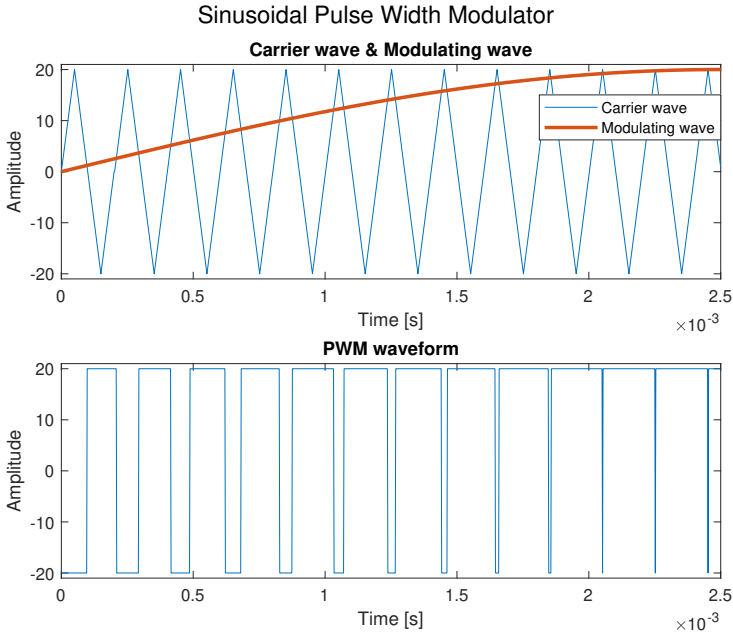


Figure 2.3: An illustration of how the PWM is generated with a carrier and a reference signal.

2.2 Field Oriented Control

The goal of using field oriented control (FOC) is to make sure that the rotor is orthogonal to the magnetic field of the stator by controlling the three phase current. The way this is done with three phases is by first measuring the position of the rotor, and then by inducing a three phase current to produce a field at the desired direction. To achieve this the three phases are via Clarke and Park transforms converted in to the dq-frame, where q is the desired direction that gives maximum torque contribution, and the d is orthogonal to q and does not give any torque contribution. This change of frame is done with help of mentioned Clarke and Park transforms and the angle θ_G (the direction of the permanent magnet), and it describes the sum of the three phase magnetic field in the two dimensions d and q that rotates with the rotor. This way the d-vector can be controlled to zero and q-vector according to the desired torque. The d-vector induces a magnetic field in the same direction as the permanent magnetic field and therefore gives no force contribution [1]. Opposed to this the q-vector induces a magnetic field where the torque contribution is at its highest and therefore most useful. With this methodology the motor is controlled with a magnetic flux that is always ideally chasing the permanent magnets by 90 degrees.

2.2.1 Clarke

Firstly the three-phase current i_{UVW} is converted into a two-axis frame (α, β) attached to the stator, this is accomplished by a Clarke transformation. The Clarke transform uses the fact that the three phase system is symmetrical and thereby reducing the 120° equally spaced frame system to a two axis stationary reference frame fixed to the stator, as seen in Figure 2.4. The sum of the UVW-axis makes up the same vector as the sum of (α, β) vectors times $2/3$ but with only a two coordinate axis instead of three [3], which is the bold black arrow denoted $\bar{i}$ in Figure 2.4.

$$\begin{bmatrix} i_\alpha \\ i_\beta \end{bmatrix} = \frac{2}{3} \times \begin{bmatrix} 1 & -\frac{1}{2} & -\frac{1}{2} \\ 0 & \frac{\sqrt{3}}{2} & -\frac{\sqrt{3}}{2} \end{bmatrix} \times \begin{bmatrix} i_U \\ i_V \\ i_W \end{bmatrix} \quad (2.2)$$

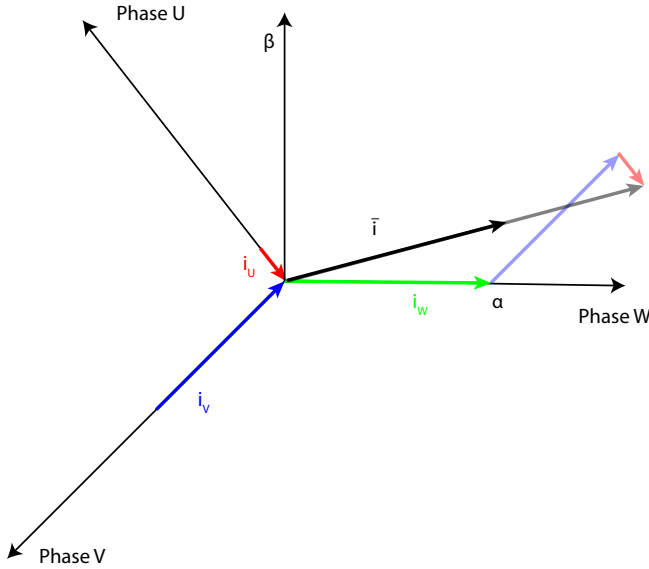


Figure 2.4: Current space vector in (α, β) coordinates, where red/green/blue is the phase-currents and the bold black arrow is the sum of the phase currents.

2.2.2 Park

Since the maximum torque is given orthogonal to the magnetic field, the direct-quadrature rotating reference frame is mostly suitable. The rotating frame fixes the currents in respect to the angle of the rotor position where the d-axis is pointing in the same direction as the rotor and the q-axis is orthogonal to it. This way the d and q currents can be controlled for maximum efficiency. This is done by minimizing the d-current since it does not provide any torque to the system, while the q-current can be used to control the motor due to it having a more or less linear relation to the torque, see equation (2.6). As seen in Figure 2.5 the (α, β) are converted to the dq-axis that follows the rotor with help of the angle θ_G [3].

$$\begin{bmatrix} d \\ q \end{bmatrix} = \begin{bmatrix} \cos\theta & \sin\theta \\ -\sin\theta & \cos\theta \end{bmatrix} \times \begin{bmatrix} \alpha \\ \beta \end{bmatrix} \quad (2.3)$$

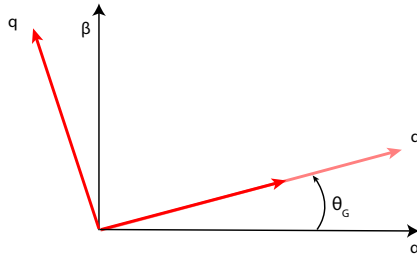


Figure 2.5: *d-q rotating frame in the (α, β) coordinate system.*

Clarke & Park transformations

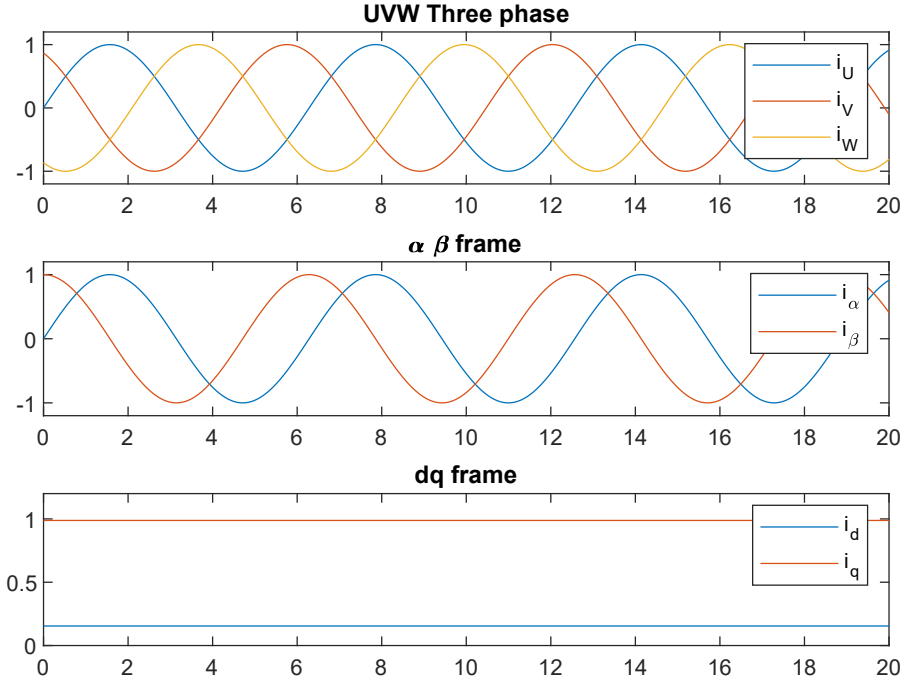


Figure 2.6: Visualization of the Clarke-Park transformations. Converting a three phase system to a static two phase.

The result of the Clarke/Park transformation in a time perspective can be seen in Figure 2.6. Where initially, in the upper plot, there is a three-phase UVW-current, the middle plot shows them transformed to a two orthogonal α and β axis fixed to the stator and lastly the bottom plot shows the current represented in a two-phase rotating reference frame (d,q). Meaning that the d-axis is fixed along the rotor and the q axis orthogonal to the rotor, and operating in this dq-frame a simple PI-controller can be used to control the d and q currents.

2.3 d-q modelling

In order to convert the three-phase system to a controllable DC system, FOC is used and Clarke/Park-transformation applied, see Section 2.2. After transformation of coordinates the system can be presented in the dq-coordinates.

The motor's stator voltages in d-q frame:

$$v_d = R_s i_d + L_d \frac{di_d}{dt} - \omega_e L_q i_q, \quad (2.4)$$

$$v_q = R_s i_q + L_q \frac{di_q}{dt} + \omega_e (L_d i_d + \psi_R), \quad (2.5)$$

where R_s , L_q and L_d are the phase resistance-and inductances, ω_e is the electrical speed. i_d and i_q are the stator currents and ψ_R is the rotors magnetic flux linkage.

The electrical torque produced by the motor:

$$T_e = \frac{3n_p}{2} (\psi_R i_q + (L_d - L_q) i_d i_q), \quad (2.6)$$

where n_p is the number of pole pairs in the motor.

Torque equilibrium:

$$J \frac{d\omega_m}{dt} = T_e - b\omega_m - T_L, \quad (2.7)$$

T_e is the electrical torque, ω_m is the mechanical speed ($\omega_e \cdot \frac{1}{n_p}$), T_L is the external load torque, b denoting the viscous damping constant, and J the inertia.

The electrical speed ω_e is the speed of the three phase electrical signals induced to the motor meaning the time it takes for the three phase voltage to fulfill a 360 degree rotation. Meanwhile the mechanical speed ω_m is the time it takes for the shaft or in this case the wheel to rotate a full rotation. The relationship between electrical and mechanical speed depends on the number of pole pairs as described in previous paragraph.

3

Open loop

This chapter describes the process of validating the motor modelled in Simulink with the physical motor. In order to confirm the Simulink model resemblance with the physical motor, the test that was made on a test rig at the corporation was also simulated and compared. Initially the motor parameters from Table 3.1 were implemented in the model and for the final result (Figure 3.5), the tuned parameters in Table 3.2 were used. The performance are *input – output* measurements, meaning a known input is applied to the system and the output is measured. An "open-loop" test is the term used in the thesis, which is also explained in the chapter.

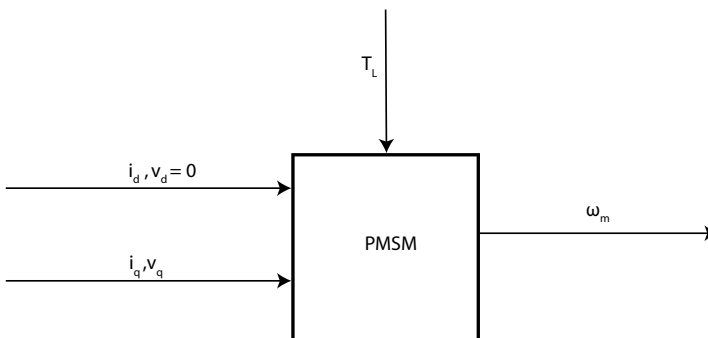


Figure 3.1: Open loop block scheme. The input to the motor-block are the dq currents and voltages as well as the torque load, output is the mechanical speed.

3.1 Model

The open loop model can be derived from equations (2.4) - (2.7) and, as illustrated in Figure 3.1, takes in the v_q and v_d voltages and the external torque that acts like a disturbance, and outputs motor speed ω_m . *Open loop* is a misleading expression since when using FOC the quadrature and direct-axis voltages are coupled with one another. It is still referred to as open loop with the assumption that the direct axis is orthogonal to the quadrature axis and therefore $i_d = 0$.

The open loop transfer function can be derived as $G_{\text{open}}(s) = \frac{Y(s)}{U(s)}$ where s is the Laplace-transform variable, which gives, from v_q to ω_m :

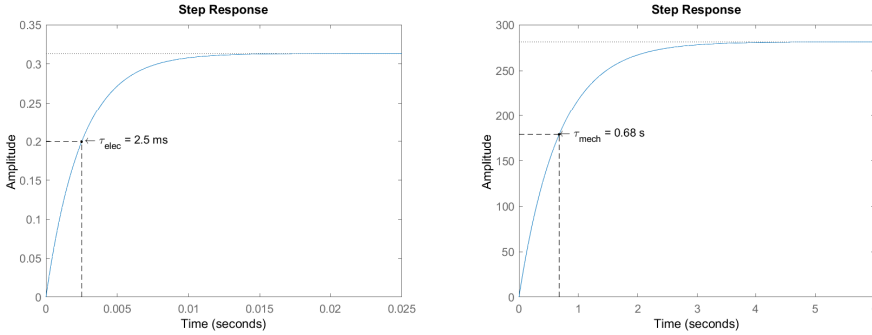
$$G_{\text{open}}(s) = \frac{\omega_m(s)}{v_q(s)} = \frac{\frac{3}{2}n_p\Psi_r}{(Js + b)(R_s + L_qs) + \frac{3}{2}n_p^2\Psi_r^2}, \quad (3.1)$$

which is a combination of the two transfer functions, $G_{\text{electrical}}(s)$ and $G_{\text{mechanical}}(s)$:

$$G_{\text{electrical}}(s) = \frac{1}{L_qs + R_s}, \quad (3.2)$$

$$G_{\text{mechanical}}(s) = \frac{\frac{3}{2}n_p\Psi_r}{Js + b}. \quad (3.3)$$

The electrical and mechanical time constants [6] are $\tau_{\text{elec}} = \frac{L_q}{R_s} = 2.5$ ms and $\tau_{\text{mech}} \approx \frac{J}{b} = 0.67$ s, which can be seen in Figure 3.2. Meaning that the electrical system is more than 270 times faster than the mechanical.



(a) Time constant for the electric system, eq 3.2).

(b) Time constant for the mechanical system, eq 3.3).

Figure 3.2: Electrical and mechanical step responses.

3.2 Parameters

The motor parameters that were given from the corporation can be seen in Table 3.1. Additional data from finite element analysis (FEA) was given as well, resulting in a 2D-map of the $L_d(i_d, i_q)$ and $L_q(i_d, i_q)$, see Figure 3.3.

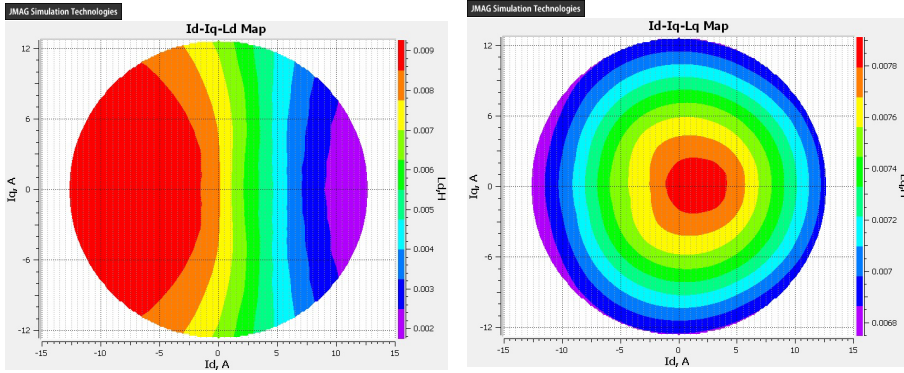
Parameter	Value	Unit	Notation
Connection type	Delta	-	-
Line resistance	2.12	Ω	R_L
Line inductance	-	mH	L_L
Phase resistance	1.633	Ω	R_s
Phase inductance	5.65	mH	L_s
Direct axis inductance	2.99	mH	L_d
Quadrature axis inductance	3.36	mH	L_q
Back emf constant line-line	95.48	Vrms/kRPM	$k_{e,L}$
Back emf constant phase	18.69	Vrms/kRPM	k_e
Pole pairs	15	-	n_p
Slot numbers	27	-	-
Mass of rotating part	1.67	kg	-
Inertia (around shaft)	53.44	kg/cm ²	J
Position sensor	4096	ppr	-

Table 3.1: Provided motor parameters.

Not all data in Table 3.1 is needed to model the motor, and some data have the wrong unit. In the next table, Table 3.2, the parameters used in the modelling are listed. The column denoted "adjusted value" is the final adjusted value that was tuned, with *trial and error method*, in order to mimic the test results from the physical motor, see discussion (Chapter 7) for more information about the adjusted values. The back emf constant is replaced with the magnetic flux linkage, since it is the parameter used in equation (2.5) - (2.6). The relationship between the magnetic flux linkage and the back emf constant is $k_e = n_p \psi_R$.

Parameter	Value	Adjusted Value	Unit	Notation
Phase resistance	1.633	3.19	Ω	R_s
Direct-axis inductance	FEA-data	-	mH	L_d
Quadrature-axis inductance	FEA-data	-	mH	L_q
Magnetic flux linkage	0.09	0.1	V · s = Weber	Ψ_R
Pole pairs	15	-	-	n_p
Inertia (around shaft)	0.005344	-	kg/m ²	J
Viscous damping	-	0.008	$\frac{\text{Nm} \cdot \text{s}}{\text{rad}}$	b
Position sensor	4096	-	ppr	-

Table 3.2: Used motor parameters.



(a) Inductance L_d as a function of i_d & i_q . (b) Inductance L_q as a function of i_d & i_q .

Figure 3.3: FEA data, from JMAG, of L_d and L_q as a function of i_d and i_q .

Since the direct/quadrature-axis inductances are non-linear functions of the direct/quadrature currents, the corporation has provided finite element analysis data from a model in a 3D-mapping program JMAG. This data has been used and implemented as lookup tables for use in the simulation models.

3.3 Validation

The validation of the motor conformity was done by an open-loop test presented in this section. As the motor was rigged with a motor-controller some modification was done in order to achieve the open-loop behaviour. The PWM modulation index was set to 1, meaning that maximum voltage, V_{DC} was the input to the PMSM. The controller was set to "torque-mode", meaning that only the current loop was controlled. The current limit was set greater than what the motor could receive, meaning that there was no limitation on the current and the motor could draw as much power as it needed. This way i_d is controlled to zero and i_q is determined by the load, motor and input voltage and acts as the "open-loop" part of the system.

Measurements

The data for the speed-torque characteristics was collected at a test rig at the corporation's facility, with a sampling frequency of 55 Hz. The PMSM was clamped to a break, which could apply a variable load torque. Speed, voltage, break-torque and current was measured for a total cycle time of around 40s. The break-torque was increased from 0 - 5.5 Nm with an increment of 0.5 Nm every 3s, the number of operating points were 12.



Figure 3.4: The test rig at the corporation, used for the data collection. The break is located at the top of the picture and is applying the load on the PMSM, which is connected to it. The three blue details are the ampere meters for each phase and the black box is the motor controller setting the "torque-mode" parameters.

Speed-torque characteristics

To evaluate the simulated model performance against the physical motor an open loop test was made. The test was made such that the no-load-speed, three phase currents and voltages and the efficiency of the motor could be obtained. The test was performed such that the reference voltage was supplied to the motor, thereafter when the speed was at steady state the load torque was gradually increased, with 0.5 Nm steps, until it reached 5.5 Nm. The RMS voltages and currents, speed-torque and efficiency can be seen in Figures 3.5 and 3.6.

Open loop model validation

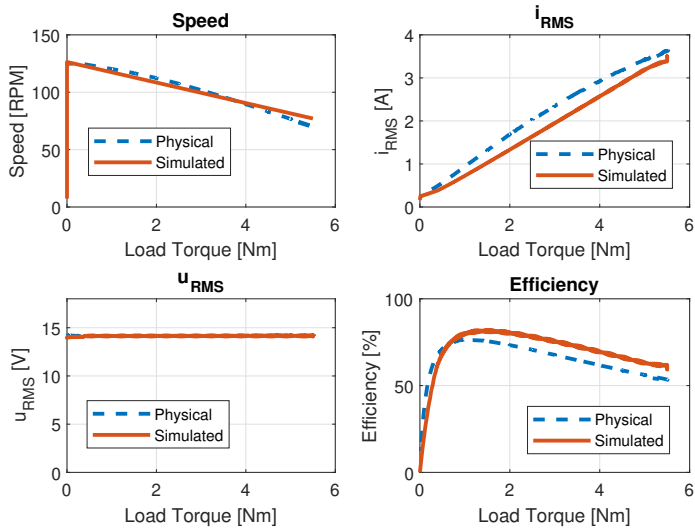


Figure 3.5: Test results with the adjusted parameters.

Open loop model validation

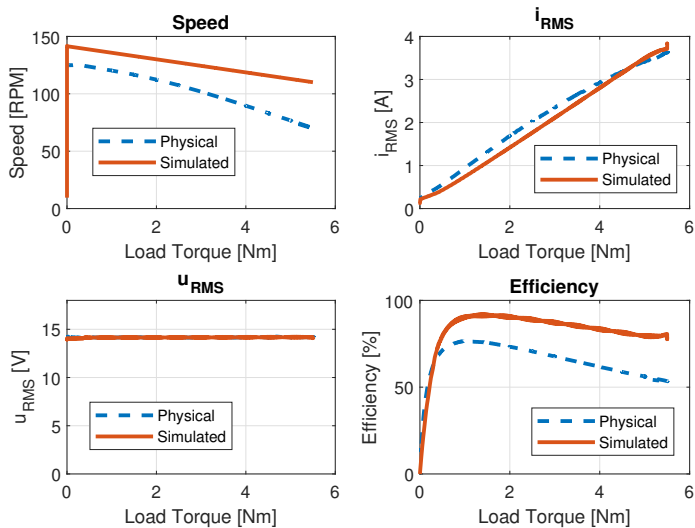


Figure 3.6: Test results with the given parameters.

In Figures 3.5 and 3.6 the efficiencies are calculated with the equation:

$$\eta = \frac{\omega_m \cdot \text{Total torque}}{\text{PF} \cdot U \cdot I},$$

where PF is the power factor, which in the simulation efficiency is neglected since it is small ($\text{PF} = \cos(\varphi) \approx 1$).

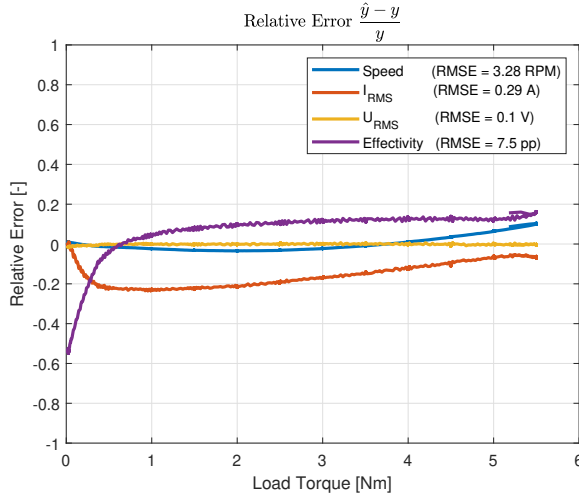


Figure 3.7: The relative error of simulated versus the physical motor, with the adjusted parameters.

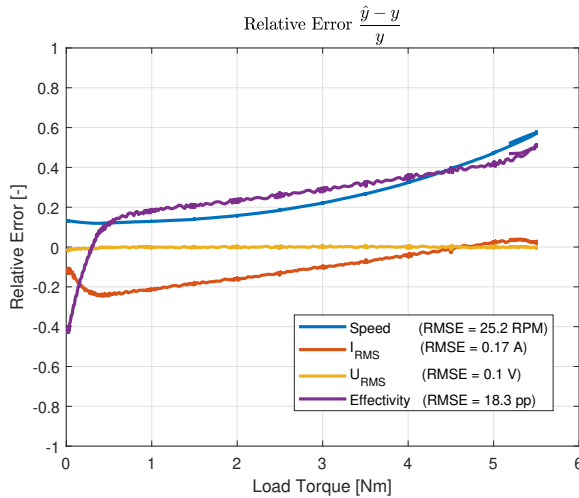


Figure 3.8: The relative error of simulated versus the physical motor, with the given parameters.

In Figures 3.7 and 3.8 the relative error is plotted for each parameter in Figures 3.5 and 3.6. The root mean square error (RMSE) is calculated as well, with the equation:

$$RMSE = \sqrt{\frac{\sum_{i=1}^n (\hat{y}_i - y_i)^2}{n}},$$

where $\hat{y}_i$ denotes the measured output from the simulated system and y_i from the physical one.

4

Filtering and Sensoring

This chapter describes the position and speed sensing used in the motor setup and how it was modelled as well as the two proposed filtering strategies EKF and UKF. The main task of the two filters is to improve measurements at lower speeds when the speed sensor update is less frequent, but also to filter any possible measurement noise. The filters are compared as a final illustration of their estimation performance.

4.1 Position sensor

The sensor used on the physical motor is a position encoder, whose output is a binary pulse. To get an interpretable signal θ the pulse needs to be decoded. The position decoder consists of three signals, A, B and Z. A and B has a certain resolution, pos_{res} , which has the unit points per revolution (ppr). The two signals are shifted by 90 degrees, seen in Figure 4.1. The offset between A and B is used to detect which direction the motor is turning. Additionally there is an index pulse, Z, which only has one pulse per revolution and is used for pulse homing and count verification.

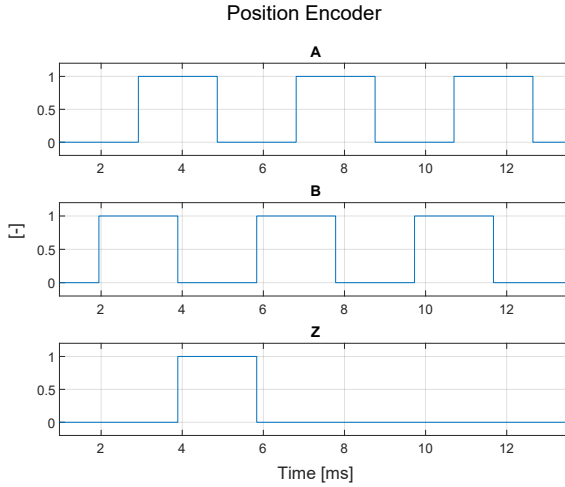


Figure 4.1: An example of an output of the position sensor.

Decoder

In order to get the wanted outputs, the angle and speed of the rotor (θ & ω), one has to decode the binary pulses with some binary mathematics. Note that this is a simplification of the decoder.

Firstly the A signal triggers a calculation block at its rising edge, with this information one can calculate the sampling frequency, and therefore the speed by subtracting current time with the time of the previous iteration.

By changing B from switching between 0 and 1 to 1 and -1, and only identify its signal at the rising edge of A, one can get the direction, which will appear either positive or negative. With the direction one can at each pulse from A add +1 or -1 and sum up the total of pulses, pos_{count} , that will be used to calculate the angle of the rotor as follows:

$$\theta = 360 \cdot \frac{pos_{count}}{pos_{res}} [^\circ]. \quad (4.1)$$

This meaning that it checks which pulse it is at momentarily, compares it with the resolution of the encoder and multiplies with number of degrees of a revolution to get its position.

It should be noted that in the modeled motor the encoder/decoder is represented with a simple function that only updates the speed and angle output of the motor given the resolution of the encoder instead of modeling an actual encoder and decoder, this gives the same results but uses less computational power. A block scheme of the sensor interpretation is found in Figure 4.2.

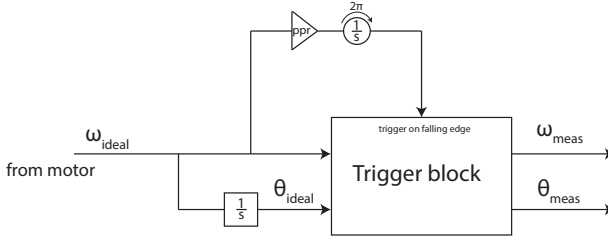


Figure 4.2: Block scheme of the position sensor.

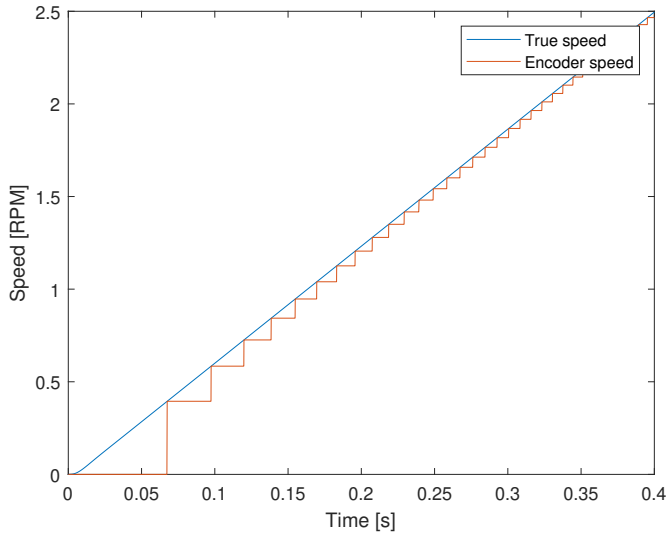


Figure 4.3: Speed from the position sensor vs true speed.

In Figure 4.3 the speed from the decoder is plotted and compared to the true speed, i.e. integrated from equation (2.7). The input is a voltage ramp in v_q with gradient 1, which corresponds to an acceleration of $\sim 0.65 \text{ rad/s}^2$, and one can see that the measurement update is less frequent for lower speeds.

4.2 Filter theory

This chapter describes the theory of the two filters investigated in this thesis, that being the UKF and EKF, and the process of generating the filter models for the motor.

When the system is non-linear, as in this case, a normal Kalman filter can not be used, and therefore there is a need for variants of the Kalman filter that can handle non-linear systems. Two of the most widely used non-linear variants are the Extended Kalman Filter (EKF) and the Unscented Kalman Filter (UKF). The Extended Kalman filter uses the Jacobian of the transition function to linearize the system at each operating point. The unscented Kalman filter uses the Unscented Transform (UT) to generate a set of sigma points with the help of the mean and covariance, and runs the points through the transition function to then re-calculate the mean and covariance of the transformed sigma points to get the estimation for the next time step.

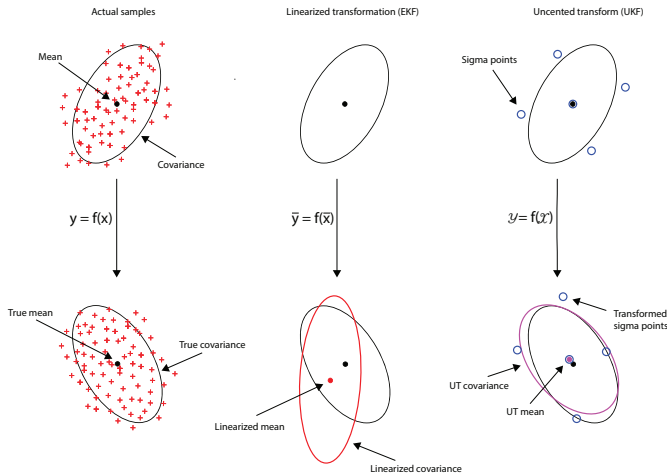


Figure 4.4: Visualization of the EKF and UKF transformations compared to actual sampling.

Figure 4.4 shows a representation of how the two filters predict the next step compared to the real value of the next step. The EKF uses a linearization of the transfer function and can therefore only change linearly, while the UKF uses the UT that can transfer each sigma point through the non-linear function and then recalculate the mean and covariance more accurately.

4.2.1 Extended Kalman Filter

As mentioned in previous section the EKF is applied to estimate the speed and angle of the rotor over time, by linearizing the system at each operating point. It is applied in order to get a more accurate state estimation to control and to filter out eventual noise.

The nonlinear system can be presented as:

$$\begin{aligned} x_k &= f(x_{k-1}, u_{k-1}) + w_{k-1}, \\ z_k &= h(x_k) + v_k, \end{aligned} \quad (4.2)$$

where x_k and z_k are the states and the outputs of the system and u is the control signal. f is the non-linear function of the system and h is the output function. w_k is the system noise and represent disturbances while the measurement noise v_k represents disturbances in the measurement signals. They are assumed Gaussian with zero mean and covariances Q_k and R_k . A great value of Q_k means greater confidence in measurements, and a small value less confidence in measurements, i.e. more in the model. Q_k and R_k do not correlate with each other. Implementing the EKF, for a fixed sampling time T_s , one can obtain the state estimate $\hat{x}_k$ and covariance matrix P_k .

The EKF consists largely of two computational steps; prediction and innovation. In the prediction step, the previous state estimates $\hat{x}_{k-1}$ with its covariances P_{k-1} is used to predict the upcoming states and covariances, as in equation (4.3) where Euler forward has been applied.

$$\begin{aligned} \hat{x}_{k|k-1} &= \hat{x}_{k-1} + T_s f(\hat{x}_{k-1}, u_{k-1}), \\ P_{k|k-1} &= J_k P_{k-1} J_k^T + Q, \end{aligned} \quad (4.3)$$

where T_s is the sampling time and $k|k-1$ is the information from prior estimate. The transition function of the nonlinear system is $f(x)$ and the Jacobian matrix is obtained as:

$$J_k = \frac{\partial(x_{k-1} + T_s f(x_{k-1}, u_{k-1}))}{\partial x_{k-1}}. \quad (4.4)$$

For the innovation step the estimated state vector $\hat{x}_k$ is corrected with help of the measurements made at the current time step, z_k , and the Kalman gain.

$$\begin{aligned} \hat{x}_k &= \hat{x}_{k|k-1} + K_k(z_k - H_k \hat{x}_{k|k-1}), \\ P_k &= P_{k|k-1} - K_k H_k P_{k|k-1}, \end{aligned} \quad (4.5)$$

where H_k is the Jacobian for the measurement matrix and K_k is the Kalman gain. Equation (4.6) defines the Jacobian of the measurement matrix and the gain, where h denotes the measurement matrix.

$$\begin{aligned} H_k &= \frac{\partial h(x)}{\partial x}, \\ K_k &= P_{k|k-1} H_k^T (H_k P_{k|k-1} H_k^T + R)^{-1}. \end{aligned} \quad (4.6)$$

The EKF computational process is summarized in Table 4.1.

Step	Equation	
Prediction	$\hat{x}_{k k-1} = \hat{x}_{k-1} + T_s f(\hat{x}_{k-1}, u_{k-1})$	Prediction step
Jacobian matrix	$J_k = \frac{\partial(x_{k-1} + T_s f(x_{k-1}, u_{k-1}))}{\partial x_{k-1}}$	
Covariance matrix	$P_{k k-1} = J_k P_{k-1} J_k^T + Q$	
Measurement Jacobian	$H_k = \frac{\partial h(x)}{\partial x}$	Innovation step
Kalman gain	$K_k = P_{k k-1} H_k^T (H_k P_{k k-1} H_k^T + R)^{-1}$	
Covariance update	$P_k = P_{k k-1} - K_k H_k P_{k k-1}$	
Correction	$\hat{x}_k = \hat{x}_{k k-1} + K_k (z_k - H_k \hat{x}_{k k-1})$	

Table 4.1: EKF computational steps.

Note that the initialization of the state vectors and covariance matrices, x_0, P_0 , are excluded in Table 4.1. These are initialization points that are to be set by the user. The state variables are the two rotor currents, the speed and position and the rotor voltages are the input:

$$\begin{aligned} x_k &= [i_d \quad i_q \quad \omega_m \quad \theta]^T \\ u_k &= [v_d \quad v_q]^T \\ y_k &= [i_d \quad i_q \quad \omega_m \quad \theta]^T \end{aligned}$$

The initial values of the state vector and covariance matrix are:

$$\begin{aligned} x_0 &= [0 \quad 0 \quad 0 \quad 0] \\ P_0 &= 10^{-2} \cdot \mathbb{I}^{4 \times 4} \end{aligned}$$

This generates, from equation (2.4) – (2.7), the transition function of the nonlinear system:

$$f(x, u) = \begin{bmatrix} -\frac{R_s}{L_d} i_d + \omega_e \frac{L_q}{L_d} i_q n_p + \frac{1}{L_d} v_d \\ -\frac{R_s}{L_q} i_q - \omega_e \frac{L_d}{L_q} i_d n_p + \frac{1}{L_q} v_q - \omega_e n_p \frac{\Psi_R}{L_q} \\ \left(\frac{3n_p}{2 \cdot J} (\Psi_R i_q + (L_d - L_q) i_d i_q) - b \omega_m - \hat{T}_L \right) / J \\ \omega_m \end{bmatrix} \quad (4.7)$$

Note that $\hat{T}_L$ is the observed load torque, see Section 5.1.2 for more details. And the Jacobians are:

$$J = \begin{bmatrix} 1 - T_s \frac{R_s}{L_d} & T_s \frac{L_q}{L_d} \omega_e n_p & T_s \frac{L_q}{L_d} & 0 \\ -T_s \frac{L_d}{L_q} \omega_e n_p & 1 - T_s \frac{R_s}{L_q} & -T_s \left(\frac{L_d}{L_q} i_d + \frac{\Psi_R}{L_q} \right) & 0 \\ \frac{3n_p}{2 \cdot J} (L_d - L_q) i_q T_s & \frac{3n_p}{2 \cdot J} (\Psi_R + (L_d - L_q) i_d) T_s & 1 - \frac{T_s b}{J} & 0 \\ 0 & 0 & T_s & 1 \end{bmatrix} \quad (4.8)$$

$$H = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (4.9)$$

H being an identity matrix meaning all states can be and are measured.

4.2.2 Unscented Kalman Filter

The UKF as opposed to EKF does not need to linearize the function at any point, but can through a set of chosen sigma points use the non-linear function that represents the system instead. This way the filter can calculate the probability distribution after the non-linear function, this process is called the Unscented Transform. Other than this the filter operates as any filter with a time update (prediction step) and a measurement update (filter step).

Before initializing the first unscented transform step an initial guess of the mean and uncertainty must be made.

$$\begin{aligned} x_{0|0} &= E(x_0), \\ P_{x,0|0} &= \|x_0 - \hat{x}_{0|0}\|^2, \end{aligned} \quad (4.10)$$

where x is the state vector

$$x = [i_d \quad i_q \quad \omega_m \quad \theta]^\top$$

The initial guesses were set to

$$\begin{aligned} x_{0|0} &= [0 \quad 0 \quad 0 \quad 0]^\top \\ P_{x,0|0} &= \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \end{aligned}$$

After the initialization the unscented transform step is implemented and sigma points are selected. Note that the sigma points are not selected randomly, but are defined by the expectation and covariance from the previous iteration. The number of sigma points depends on the number of states ($2n + 1$ sigma points can be obtained).

The sigma points are selected as follows, the first one is the mean, and the rest are calculated with the mean $\pm$ the rows of the matrix $\sqrt{nP_{\hat{x},k-1}}$. Where n is the number of states. This means e.g. that the second sigma point is the mean plus the first row of the square root, the third point uses the second row and so on, and then the same thing is done again but minus each row. Since the covariance is a 4x4 matrix the square root of the covariance matrix times a constant will also

result in a 4x4 matrix, thereby using the mean by itself and then $\pm$ each row will give a total of nine sigma points.

$$\begin{aligned}\sigma_{k-1}^i &= \hat{x}_{k-1} \quad \text{for } i = 0, \\ \sigma_{k-1}^i &= \hat{x}_{k-1} + [\text{row}_i(\sqrt{nP_{\hat{x},k-1}})]^\top \quad \text{for } i = 1, \dots, n, \\ \sigma_{k-1}^i &= \hat{x}_{k-1} - [\text{row}_{i-n}(\sqrt{nP_{\hat{x},k-1}})]^\top \quad \text{for } i = n+1, \dots, 2n.\end{aligned}\tag{4.11}$$

In this operation there is a need to take the square root of a matrix, the square root of a matrix is any matrix that satisfies $\sqrt{A}^\top \sqrt{A} = A$.

Note that there are a number of matrices that will satisfy this property as $\sqrt{A}^\top Q^\top Q \sqrt{A} = A$ for any Q where $Q^\top Q = I$.

For this calculation Cholesky Factorization was used to obtain the square root since it is both numerically efficient and stable [11].

In two dimensions with $n = 2$ the sigma points can be illustrated as in Figure 4.4.

The Sigma points are then processed though the non-linear function using the Euler forward method as following:

$$\sigma_{k|k-1}^i = \hat{x}_{k-1} + T_s f(\sigma_{k-1}^i, u_{k-1}),\tag{4.12}$$

where f is the non-linear function (4.7), T_s the sampling time and $u = [v_d \quad v_q]^\top$ the measured voltages.

The predicted estimate of the states are a sample average of the transformed sigma points. The difference between each transformed sigma point and their sample averages are used to get the covariance matrix of the predicted states.

$$\begin{aligned}\hat{x}_{k|k-1} &= \sum_{i=0}^{2n} W^i \sigma_{k|k-1}^i, \\ P_{\hat{x},k|k-1} &= \sum_{i=0}^{2n} W^i (\sigma_{k|k-1}^i - \hat{x}_{k|k-1})(\sigma_{k|k-1}^i - \hat{x}_{k|k-1})^T + Q.\end{aligned}\tag{4.13}$$

Where W^i is a weight that can be assigned to each sigma point, in this case they are all equally weighted making $W^i = \frac{1}{\# \text{ of } \sigma\text{-points}} \forall i$.

Q is the measurement noise covariance matrix, and as with the EKF a larger value of Q gives more confidence in the measurement and vice versa.

There is an optional sigma point update step that can be implemented here. It uses the calculated sample averages and covariance matrices from (4.13) to compute new sigma points that will have a more accurate covariance. Because of this step adding complexity and computational power without significant contribution this step is not considered in this application [15].

The measurement update step is then calculated with the transformed sigma points from equation (4.12) and measurement matrix h being an identity matrix as in (4.9). The predicted values of the measurements are then calculated the same way as the predicted values of the states.

$$\begin{aligned} s_{k|k-1}^i &= h(\sigma_{k|k-1}^i), \\ \hat{y}_{k|k-1} &= \sum_{i=0}^{2n} W^i s_{k|k-1}^i. \end{aligned} \quad (4.14)$$

The covariance of the predicted measurement is calculated in the same way as the covariance of the predicted states. A covariance matrix between the predicted states and their means, and the predicted measurement and its mean is also calculated.

$$\begin{aligned} P_{\hat{y}} &= \sum_{i=0}^{2n} W^i (s_{k|k-1}^i - \hat{y}_{k|k-1})(s_{k|k-1}^i - \hat{y}_{k|k-1})^T + R, \\ P_{\hat{x}\hat{y}} &= \sum_{i=0}^{2n} W^i (\sigma_{k|k-1}^i - \hat{x}_{k|k-1})(s_{k|k-1}^i - \hat{y}_{k|k-1})^T, \end{aligned} \quad (4.15)$$

where R is the process noise covariance matrix.

The Kalman gain is calculated as $K_k = P_{\hat{x}\hat{y}} P_{\hat{y}}^{-1}$ and with that the filters final state estimate and covariance matrix are calculated with:

$$\begin{aligned} \hat{x}_{k|k} &= \hat{x}_{k|k-1} + K_k (y_k - \hat{y}_{k|k-1}), \\ P_{\hat{x},k|k} &= P_{\hat{x},k|k-1} - K_k P_{\hat{y}} K_k^T. \end{aligned} \quad (4.16)$$

Filter comparison

In Figure 4.5 the two filters are compared. The stair-like signal (ω_{meas}) is the measurement update provided from the position sensor. The dotted line is the true mechanical speed of the motor (ω_{true}), from equation (2.7) and the blue and red lines are estimated speeds with the extended and unscented Kalman filters.

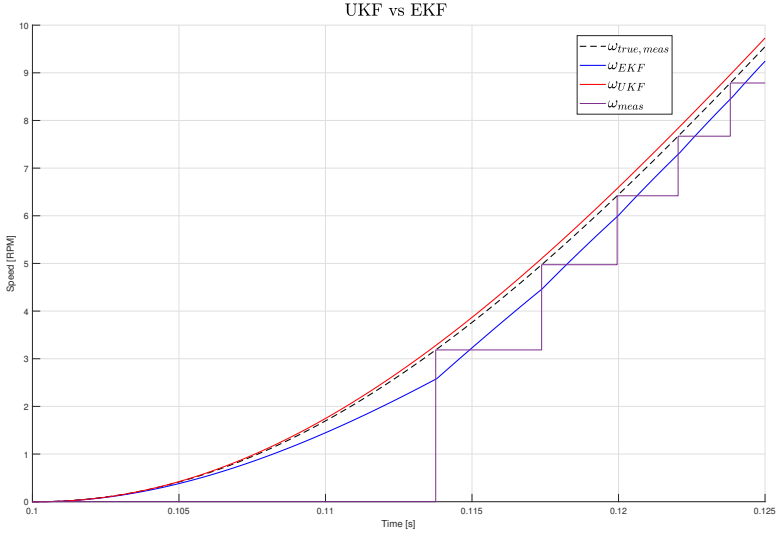


Figure 4.5: Speed estimation showing the speed measurement, true speed and estimated speed with EKF and UKF.

One can see that the measurement update is slow at low speeds, both estimated speeds from the filters are more close to the true speed of the motor than the measurement update. As can be seen the UKF performs better than the EKF and therefore the UKF is used for further implementation in the control model. The Q and R matrices that can be used to tune the filters and were set as following, where a greater value on the two first diagonal elements in Q represents that the filters trust more in the current measurements than angle and speed.

$$Q = \begin{bmatrix} 1000 & 0 & 0 & 0 \\ 0 & 1000 & 0 & 0 \\ 0 & 0 & 10 & 0 \\ 0 & 0 & 0 & 10 \end{bmatrix}, \quad R = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

5

Controlling

This chapter explains what the control system and its control loops look like, then a more theoretical description of each control strategy is found followed by illustrative block schemes of how the different setups were implemented. In Appendix A.3 the different setups and models from simulink are found. The control system of the motor consist of three control loops in a cascade coupling:

- Outer loop, which controls the speed ω_m of the motor.
- Inner loop, which controls the current of the quadrature axis i_q , which in turn controls the electromagnetic torque.
- An additional inner loop along side i_q , which is to minimize the current of the direct axis i_d .

In Figure 5.1 the control system is illustrated.

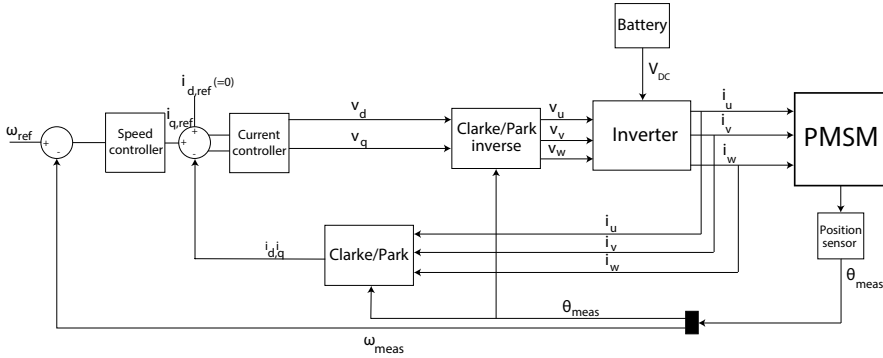


Figure 5.1: An overview of the control system.

5.1 Control theory

This chapter describes the theory of the two different control strategies used, PI and MPC. It also describes discretization, anti-windup, cascade-coupling, tuning of the PI-controllers and the implementation of a load torque observer.

5.1.1 PI

A general PI-controller can be described by the equations:

$$u(t) = K_p e(t) + K_I \int_0^t e(\tau) d\tau, \quad (5.1)$$

$$\text{where, } e(t) = r(t) - y(t), \quad (5.2)$$

where u is the control signal (input to the control system) and $e(t)$ is the control error defined as the difference between the reference signal and the output of the system, which is intended to be minimized.

The proportional part of a PI-controller is controlling the speed of the system where a greater K_p results in a faster response. A P-regulator cannot by itself eliminate the control error but often results in a static error. Therefore, it is natural to add an integral whose purpose is to gain the input signal as long as the output is less than the reference. Intuitively the greater values of K_p and K_I the faster rise to the desired output, but one has to bare in mind that the signal is getting more oscillative with a greater amplitude and eventually will lead to instability [6].

Discrete

Unlike the motor, which is running in continuous time, the controlling of the system would be done digitally by a microcontroller that is installed with the motor setup on the robot. Because of this the controller needs to be presented in discrete time. In order to discretize equation (5.1) one can replace the integral with an accumulated sum and multiply it with the time step, T_s [6].

$$u[k] = K_p e[k] + K_I \sum_{n=1}^k e[n] T_s. \quad (5.3)$$

Anti-windup

Since there are limitations on the voltage input to the motor, V_{DC} , a saturation is needed, which means when these are reached the voltage will saturate and not exceed these physical limitations. The saturation is a problem when having an integral in the controller because the PI-controller is not provided any information whether the voltage is saturated or not and will always keep accumulating the error. When the voltage is saturated the integral part of the regulator will continue to accumulate the error without being able to increase the control signal. This can result in problems when the error changes direction, due to previously accumulated error the regulator will continue with the same maximum input signal. This is called an integrator windup and can be mitigated with the help of an anti-windup system, the clamping anti-windup strategy is used in this work which is also referred to as the conditional anti-windup. The clamping anti-windup is performed by adding switching elements to an integral part of the PI-controller. The function of switching elements is to stop or restrict the integration when the controller signal are exceeding a predetermined bound of saturation which in turn will help in minimizing the windup phenomenon [10].

Saturation equation:

$$u[k]_{sat} = \begin{cases} V_{DC}, & \text{if } u[k] > V_{DC}, \\ -V_{DC}, & \text{if } u[k] < -V_{DC}, \\ u[k], & \text{otherwise.} \end{cases} \quad (5.4)$$

To solve the integrator windup, firstly check if the signal is saturated:

$$s[k]_{1|0} = \begin{cases} 1, & \text{if } u[k]_{sat} - u[k] = 0, \\ 0, & u[k]_{sat} - u[k] \neq 0. \end{cases} \quad (5.5)$$

Secondly, compare $e[k]$ with $u[k]$ to know if the integrated part is increasing or decreasing:

$$e[k]_{1|0} = \begin{cases} 0, & \text{if } e[k] \cdot u[k] < 0, \\ 1, & \text{if } e[k] \cdot u[k] \geq 0. \end{cases} \quad (5.6)$$

Compare both cases in order to know when to accumulate the error:

$$\text{int}_{1|0} = \begin{cases} 1, & \text{if } s[k]_{1|0} \& e[k]_{1|0} = 1, \\ 0, & \text{else.} \end{cases} \quad (5.7)$$

Cascade

Cascade controlling is a methodology that uses one or several extra measurement signals in order to calculate the control signal [5]. The cascade controller for the motor consists of three control loops as mentioned in the beginning of this chapter. The inner secondary loops, which are controlling the currents i_d and i_q are faster than the outer primary loop that controls the speed ω_m . This is a necessity if it is to be worthwhile to use cascade control. In Figure 5.2 a typical cascade system is illustrated, F2 and G2 constitutes the current regulations whilst F1 and G1 the speed controller. The measurable signal z , the current drawn by the motor, is fed back to controller F2 and hence eliminate the error faster. This is a major benefit with cascade control, disturbances of subsystem G2 is immediately taken care of by the secondary loop.

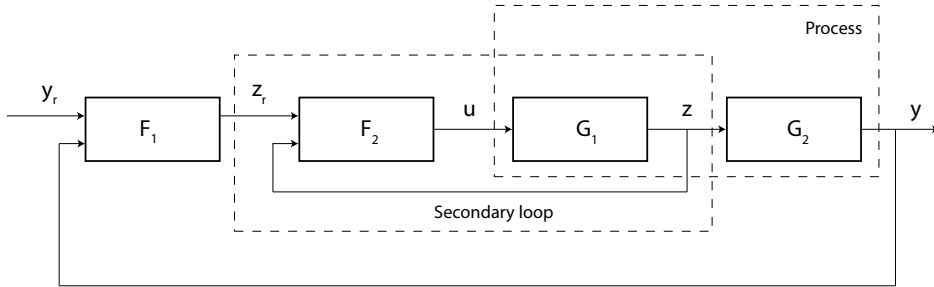


Figure 5.2: Block scheme of a cascade control loop. The outer loop, $F1 \rightarrow G2$, is the primary one.

Tuning

The tuning of the PI-controllers were made using the Direct Synthesis Method, which is based on a process model and a desired closed-loop transfer function where the time constant of the closed-loop system is a design parameter. It is applied in [17], which implies it is an easy method that offers accuracy of settling time according to the desired one. As mentioned in previous section, when having a cascade couple system the inner loop is tuned first followed by the outer loop.

Inner loop

Assuming $i_d = 0$ the transfer function $G_{e,inner}(s)$ from v_q to i_q is as following:

$$G_{e,inner}(s) = \frac{1}{L_q s + R_s}. \quad (5.8)$$

The ideal closed loop, $G_{cc,inner}(s)$, should be:

$$G_{cc,inner}(s) = \frac{\frac{1}{\tau_{c,i}s}}{1 + \frac{1}{\tau_{c,i}s}}, \quad (5.9)$$

where $\tau_{c,i}$ is the desired time constant of the closed-loop system. In steady state, ideally, there should be no control error since $G_{cc,inner}(0) = 1$. The transfer function for a closed loop system with a controller $F_{c,inner}(s)$ is:

$$G_{cc,inner}(s) = \frac{F_{c,inner}(s)G_{e,inner}(s)}{1 + F_{c,inner}(s)G_{e,inner}(s)}. \quad (5.10)$$

Meaning that if $F_{c,inner}(s)G_{e,inner}(s) = \frac{1}{\tau_{c,i}s}$ the desired system is obtained. This generates:

$$F_{c,inner}(s)(s) = \frac{1}{\tau_{c,i}s} \quad G_{e,inner}^{-1}(s) = \frac{1}{\tau_{c,i}s}(sL_q + R_s), \quad (5.11)$$

$$\text{i.e., } k_{p,iq} = \frac{L_q}{\tau_{c,i}}, \quad k_{i,iq} = \frac{R_s}{\tau_{c,i}}, \quad (5.12)$$

$$\text{and since } L_d \approx L_q \implies k_{p,id} \approx k_{p,iq}, \quad k_{i,id} \approx k_{i,iq}. \quad (5.13)$$

A relationship between settling time $t_{s,i}$ for reference tracking and $\tau_{c,i}$ is

$$T_{s,i} = 4\tau_{c,i}. \quad (5.14)$$

$T_{s,i}$ is the time it takes for the response to reach and stay within 2% of it's final value. Choosing $t_{s,i}$ to 1 ms gives a $\tau_{c,i} = 0.25$ ms. The tuning result for the inner loop is shown in Figure 5.3 where a step response is made and one can see that the wanted rise time matches the simulated one well.

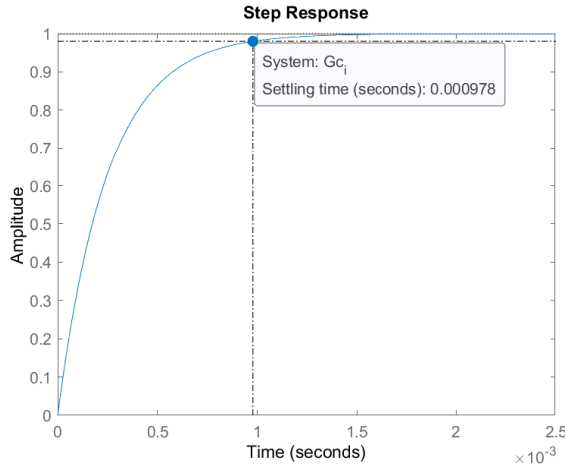


Figure 5.3: A step response for the inner closed-loop.

Outer loop

The transfer function $G_{e,outer}(s)$ from v_q to the mechanical speed ω_m [rad/s] is:

$$G_{e,outer}(s) = \frac{B_b}{A_a C_c + B_b \Psi_R}, \quad (5.15)$$

where

$$A_a = sJ + b, \quad (5.16)$$

$$B_b = \frac{3}{2} n_p \Psi_R, \quad (5.17)$$

$$C_c = sL_q + R_s, \quad (5.18)$$

which gives the controller

$$F_{c,outer}(s) = \frac{1}{\tau_{c,o}s} \left(\frac{A_a C_c + B_b \Psi_R}{B_b} \right), \quad (5.19)$$

$$\text{i.e., } k_{p,\omega_m} = \frac{1}{\tau_{c,o}} \frac{JR_s + bL_q}{B_b}, \quad k_{i,\omega_m} = \frac{1}{\tau_{c,i}} \left(\frac{bR_s + B_b^2}{B_b} \right). \quad (5.20)$$

5.1.2 Model Predictive Control

Model predictive control (MPC) is a controller that is based on optimization. The online optimization, stability and performance are mostly understood when it comes to linear models. For the non-linear systems there has been improvements as well but some questions persists, including the performance of the online computation [16].

Because of the cascade coupling of the system there are several options of the model predictive control. The options are model predictive speed controller (MPSC), model predictive current controller (MPCC) and model predictive speed and current controller (MPSCC). In [19], all have been investigated and the implementation of them can be read there. A simplified MPSCC is implemented, meaning that the d-axis current is assumed to be zero in the modelling of the controller, but it is in fact controlled with a PI. The external load torque is a disturbance and it will be taken care of with an observer, which estimates the load and is thereafter interpreted as a state in the MPC.

The main idea of a MPC is, with information about the plant and the prior information of the control input, predict the trajectory of the states for a certain horizon N and calculate the optimal control inputs to get there. This makes it an optimal control problem, OCP. To solve the OCP CasADi is used, which is an open-source tool for nonlinear optimization and algorithmic differentiation [2], and the solver for the problem is Ipopt.

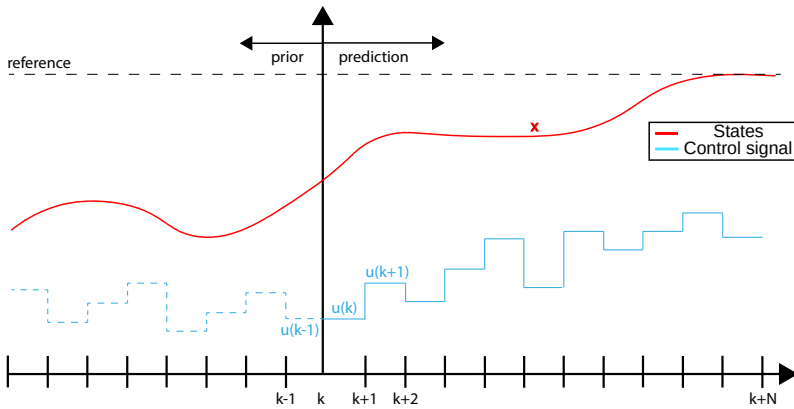


Figure 5.4: MPC principle of working.

Model

Assuming a PI-controller is regulating the direct current i_d to zero the expression of the model, equations (2.4)-(2.7), is:

$$\begin{bmatrix} \frac{di_q}{dt} \\ \frac{d\omega_m}{dt} \\ \frac{dT_L}{dt} \end{bmatrix} = \begin{bmatrix} -\frac{R_s}{L_q} & -\frac{\Psi_R n_p}{L_q} & 0 \\ \frac{3n_p \Psi_R}{2J} & -\frac{b}{J} & -\frac{1}{J} \\ 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} i_q \\ \omega_m \\ T_L \end{bmatrix} + \begin{bmatrix} \frac{T_s}{L_q} \\ 0 \\ 0 \end{bmatrix} v_q. \quad (5.21)$$

The states are $x(k) = [i_q(k) \quad \omega_m(k) \quad T_L(k)]^\top$ where T_L is an observed state. The output of the system is $y(k) = [i_q(k) \quad \omega_m(k)]^\top$, i.e. $C = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}$. The control input is the quadrature voltage, $u(k) = v_q(k)$. The constraint that need to be handled is mainly on the voltage v_q but boundaries on the current, speed and load torque is set as well, since information about the operating region is available.

$$\begin{aligned} |v_q| &\leq V_{DC}, \\ |i_q| &\leq I_{max}, \\ |T_L| &\leq T_{Lmax}, \\ |\omega_m| &\leq \omega_{mmax}. \end{aligned} \quad (5.22)$$

Runge Kutta

The discretization method used for the MPC is the fourth-order Runge-Kutta algorithm, since it has a greater stability region than forward Euler [13] and is also more accurate [18]. The calculation steps for the predicted output of the next sampling time $k + 1$, where k is the sampling instant and h is the sampling time for the Runge-Kutta, is as following:

$$x(k+1) = x(k) + \frac{h}{6}(\mathbf{k}_1 + 2\mathbf{k}_2 + 2\mathbf{k}_3 + \mathbf{k}_4), \quad (5.23)$$

$$y(k) = g(x(k), u(k)), \quad (5.24)$$

where

$$\mathbf{k}_1 = f(x(k), u(k)), \quad (5.25)$$

$$\mathbf{k}_2 = f(x(k) + \frac{h}{2}\mathbf{k}_1, u(k)), \quad (5.26)$$

$$\mathbf{k}_3 = f(x(k) + \frac{h}{2}\mathbf{k}_2, u(k)), \quad (5.27)$$

$$\mathbf{k}_4 = f(x(k) + h\mathbf{k}_3, u(k)). \quad (5.28)$$

Objective function

The goal of the MPC is to keep speed as close to the reference speed as possible, while not violating any constraints. This is achieved by minimizing an objective function and thus finding the optimal control input for the system. The formulation of the objective function is:

$$\min_{\Delta u} J(x, \Delta u) = \sum_{i=0}^{N-1} \left(\mathbf{Q} \left\| (y_{ref}(k) - y(k+i)) \right\|_2^2 + \mathbf{R} \left\| \Delta u(k+i) \right\|_2^2 \right) + \mathbf{Q}_N \left\| (y_{ref}(k) - y(k+N)) \right\|_2^2,$$

s.t.

$$x(k+1) = x(k) + \frac{h}{6}(\mathbf{k}_1 + 2\mathbf{k}_2 + 2\mathbf{k}_3 + \mathbf{k}_4), \quad (5.29)$$

$$y(k) = Cx(k), \quad (5.30)$$

$$\Delta u(k+i) = u(k+i) - u(k), \quad (5.31)$$

$$x_{min} \leq x(k+i) \leq x_{max}, \quad (5.32)$$

$$u_{min} \leq u(k+i) \leq u_{max}, \quad (5.33)$$

$$x(0) = [0 \quad 0 \quad 0]^\top, \quad (5.34)$$

$$u(0) = [0 \quad 0 \quad 0]^\top, \quad (5.35)$$

$$i = 1, 2, \dots, N-1,$$

where $y_{ref}(k) = [i_{q,ref}(k) \quad \omega_{m,ref}(k)]^\top$, the reference current is the current of previous time step $i_{q,ref}(k) = i_{q,ref}(k-1)$ and $\omega_{m,ref}(k)$ is the wanted reference speed. $\mathbf{Q}$, $\mathbf{Q}_N$ and $\mathbf{R}$ are weighting matrices for the states and the input, N is the number of predictions. The cost function accumulates the errors of the predicted outputs and the rate of the control input for $N-1$ steps and calculates the minimum input rate Δu to get to $y(k+N)$. It applies the first control input to the system, $u(0)$, and then reiterates the accumulation for the next upcoming time step and so on.

Load torque observer

Since the load torque is an external disturbance source it cannot be measured. In order to use it as a state in the MPC it needs to be observed, which is feasible since both the speed and electrical torque can be measured and calculated. Based on equations (2.6)-(2.7) the system can be expressed as:

$$\begin{pmatrix} \frac{d\omega_m}{dt} \\ \frac{dT_L}{dt} \end{pmatrix} = \underbrace{\begin{pmatrix} -\frac{b}{J} & -\frac{1}{J} \\ 0 & 0 \end{pmatrix}}_A \begin{pmatrix} \omega_m \\ T_L \end{pmatrix} + \underbrace{\begin{pmatrix} \frac{1}{J} \\ 0 \end{pmatrix}}_B T_e, \quad y = \underbrace{\begin{bmatrix} 1 & 0 \end{bmatrix}}_C x, \quad (5.36)$$

$x = [\omega_m \quad T_L]^\top$, $u = T_e$ and $y = \omega_m$. Compared to change in speed the variation of load torque is very slow and therefore its derivative is assumed to be zero.

$$\frac{dT_L}{dt} = 0. \quad (5.37)$$

The system is linear and a linear observer can be composed with the information found in [6].

$$\dot{\hat{x}} = A\hat{x} + Bu + K(y - C\hat{x}), \quad (5.38)$$

and the estimation error is

$$\tilde{x} = x - \hat{x} \implies \dot{\tilde{x}} = (A - KC)\tilde{x}. \quad (5.39)$$

For this system:

$$\begin{bmatrix} \frac{d\hat{\omega}_m}{dt} \\ \frac{d\hat{T}_L}{dt} \end{bmatrix} = \begin{bmatrix} -\frac{b}{J} & -\frac{1}{J} \\ 0 & 0 \end{bmatrix} \begin{bmatrix} \hat{\omega}_m \\ \hat{T}_L \end{bmatrix} + \begin{bmatrix} \frac{1}{J} \\ 0 \end{bmatrix} T_e + \begin{bmatrix} K_1 \\ K_2 \end{bmatrix} \left(\omega_m - \begin{bmatrix} 1 & 0 \end{bmatrix} \begin{bmatrix} \hat{\omega}_m \\ \hat{T}_L \end{bmatrix} \right), \quad (5.40)$$

where $K = [K_1 \quad K_2]^\top$ is the observer gain matrix, it can be chosen such that the matrix $A-KC$ get arbitrarily placed eigenvalues, hence the estimation error can be made to decrease towards zero arbitrarily quickly. The discrete expression, with sampling time h of equation (5.40) is:

$$\begin{aligned} \hat{\omega}_m(k+1) &= \hat{\omega}_m(k) + h \left(-\frac{b}{J} \hat{\omega}_m(k) - \frac{1}{J} \hat{T}_L(k) + \frac{1}{J} T_e + K_1(\omega_m(k) - \hat{\omega}_m(k)) \right) \\ \hat{T}_L(k+1) &= \hat{T}_L(k) + h \left(K_2(\omega_m(k) - \hat{\omega}_m(k)) \right) \end{aligned} \quad (5.41)$$

5.2 Implementation

This chapter describes how the different control and filter settings are implemented, illustrative with block schemes.

5.2.1 PI implementation

The block schedule for the PI-controller can be seen in Figure 5.5. The inverter is marked in red just to illustrate where in the setup it is suppose to be, but as mentioned in Section 2.1.1 it is negligible.

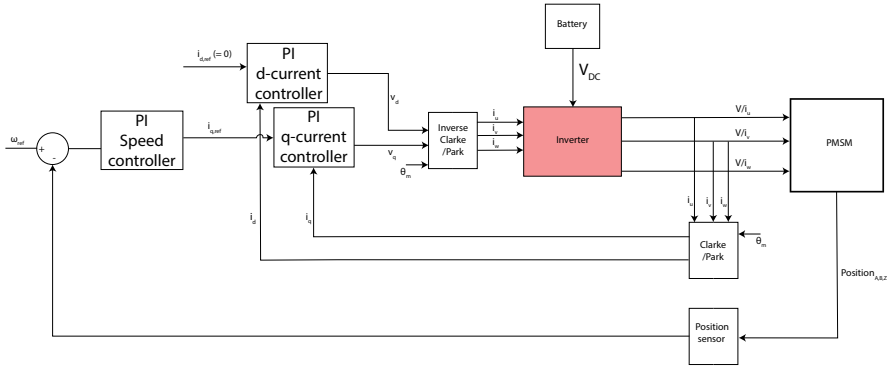


Figure 5.5: Block schedule of the system with PI-controllers.

The speed controller takes the speed reference $r = \omega_{ref}$ given from the robot and compares it with the measured speed $y = \omega_m$ to give the error $e = r - y$ and with a PI-controller this gives a control signal u to the faster, inner loop, that in turn controls the current ($u = i_{ref}$) (which is more or less linear to the torque, and therefore acts like a torque controller). The PI controller for the d-current regulates the current to zero. θ_m is the angle of the rotor and is measured in the position sensor and its signal should come from that block, these arrows are removed to make the schemes easier to interpret.

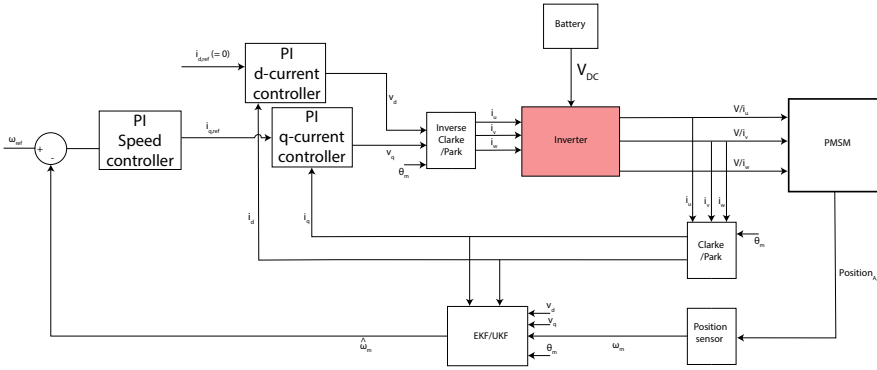


Figure 5.6: Block schedule of the system with PI-controllers and an UKF/EKF.

In Figure 5.6 the Kalman filters are implemented and the main difference is that the speed is now filtered and estimated. This means that the angle θ_m is also estimated and it is the estimated angle that is inserted in the Clarke/Park transformations and fed back to the PI-controller.

5.2.2 Model Predictive Control implementation

In Figure 5.7 the MPC implementation is illustrated. Compared to the PI setup the mechanical speed and q-current controller is merged into the MPC-block, where the optimized control signal v_q is calculated. As the load torque is observed in this setup an *observer* block is implemented as well. The signal from the inverter block to the PMSM is called V/I_{uvw} because the voltage sent to the motor and the power drawn from it occurs in the same power cable.

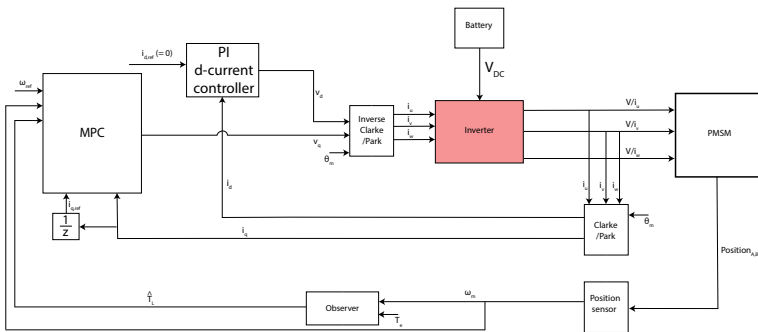


Figure 5.7: Block schedule of the system with a MPC.

In Figure 5.8 the Kalman filters are implemented with the MPC, the estimated speed from the filters are used in the Clarke/Park, observer and MPC instead of the measured speed from the position sensor.

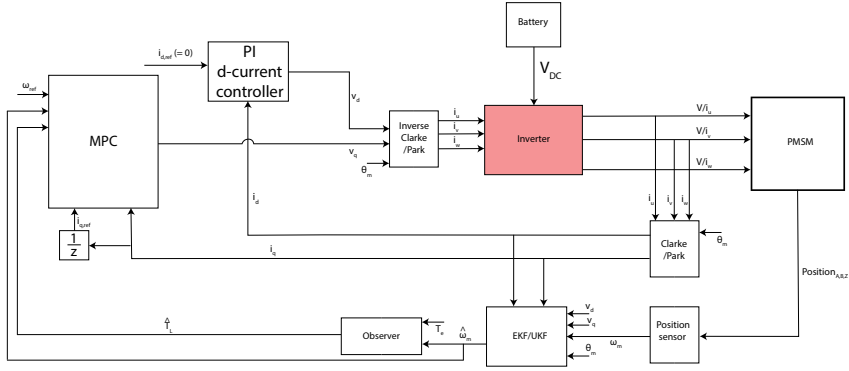


Figure 5.8: Block schedule of the system with a MPC and an UKF/EKF.

6

Results

In this chapter the results of the different controller-setups are presented. To evaluate the setups a number of tests were performed. The tests performed were based on requirements from the corporation, see Table A.1 in appendix for more information. All parameters used for the different controllers and filters are presented in tables found in Appendix A.2.

All the test performed are named as following:

- **Test 1 - Max and min speed with max torque.**
Which tests the motor controller performance of maximum and minimum speed with increasing and decreasing torque within the operational region.
- **Test 2 - Smooth start and stops.**
Which tests the motor controller performance when the motor is doing a smooth start and stop.
- **Test 3 - Speed torque.**
This test is similar test to test 2 but three different speeds are tested with maximum operating torque.
- **Test 4 - Worst case.**
This test represent a worst case scenario where the load torque reaches 8 Nm, a scenario that represents the situation when the robot is stuck at a slope resulting in maximum expected torque - 8 Nm.

6.1 Test 1 - Max-min speed & max torque

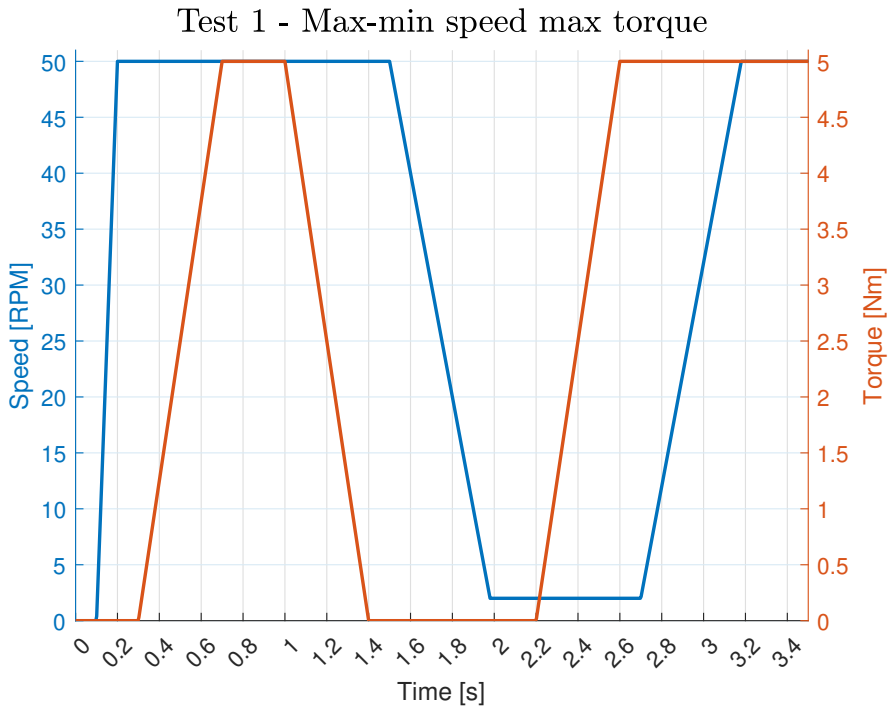


Figure 6.1: Reference speed and applied load torque for test 1.

Test 1, is shown in Figure 6.1. At 0.1s the reference speed is a ramp up to 50 RPM, thereafter the load torque is applied, at 0.3, up to 5 Nm. The load torque is thereafter removed, the speed is slowed down to 2 RPM at 1.7s and thereafter the torque is a ramp up to 5 Nm again while the speed remains at 2 RPM. Lastly the speed is a ramp up to 50 RPM while the torque is kept constant at 5 Nm. This test is performed in order to evaluate the controllers for the minimum and maximum speed, increasing and decreasing the max operating torque within the operational region.

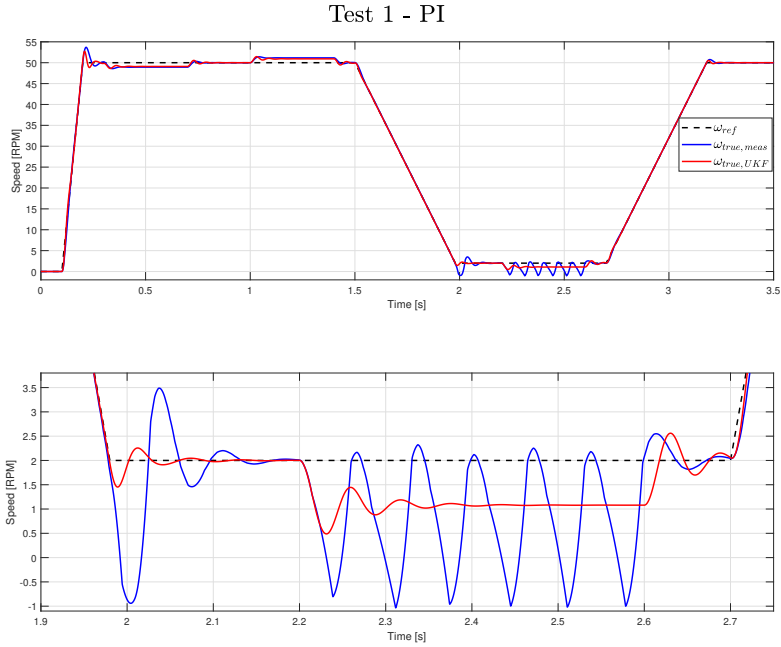


Figure 6.2: PI-controlled motor for test 1.

From the results of the the PI-controller on test 1 (Figure 6.2) it can be seen that when under load in low speeds, at ~ 2 s, the controller that uses the direct encoder measurement $\omega_{true, meas}$ starts to oscillate due to the low resolution of the position measurement at low speeds, while the controller that uses estimated measurements from the UKF performs better. Note that with an UKF the motor does not start reversing due to the external torque load input, but the speed drops to 1 RPM. The bottom subfigure displays a zoomed part of the upper subfigure, at low speed (2 RPM).

Test 1 - MPC

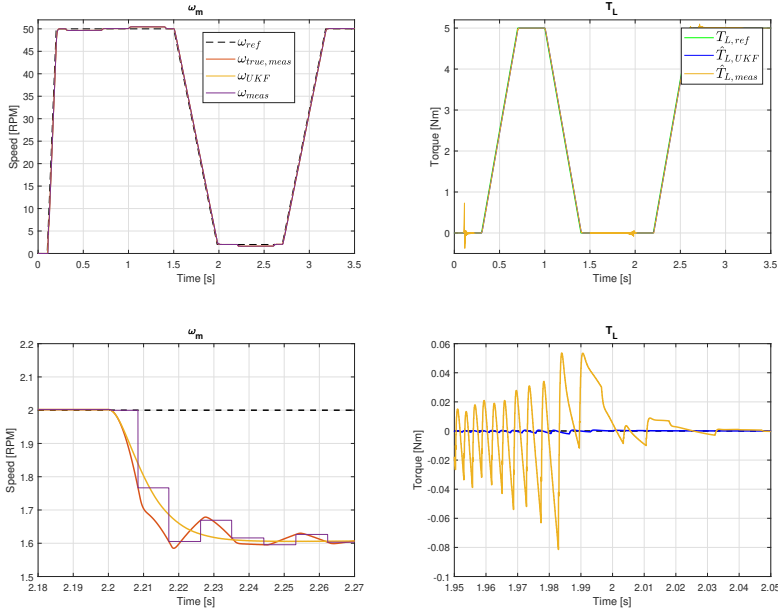


Figure 6.3: MPC-controlled motor with and without an UKF.

In Figure 6.3 test 1 was performed with the MPC, with and without the unscented Kalman filter. The two upper subfigures displays the speed and torque behaviour, while the bottom are zoomed parts of the upper subfigures at low speed. The signal denoted $\omega_{true, meas}$ is the true motor speed when using the encoder measurement as output, in the torque plot the observed torque $\hat{T}_{L, meas}$ is achieved when using this speed. ω_{UKF} is the estimated speed used when the UKF is added in the MPC-setup, ω_{UKF} is then also used as input to generate the observed torque observer $\hat{T}_{L, UKF}$. In the bottom left subfigure one can see that the speed does not drop more than ~ 0.5 RPM when any load is applied. The observed torque is significantly more stable at acceleration and deceleration with the estimated speed from the UKF used as system output, this can be observed in the bottom right subfigure (blue line).

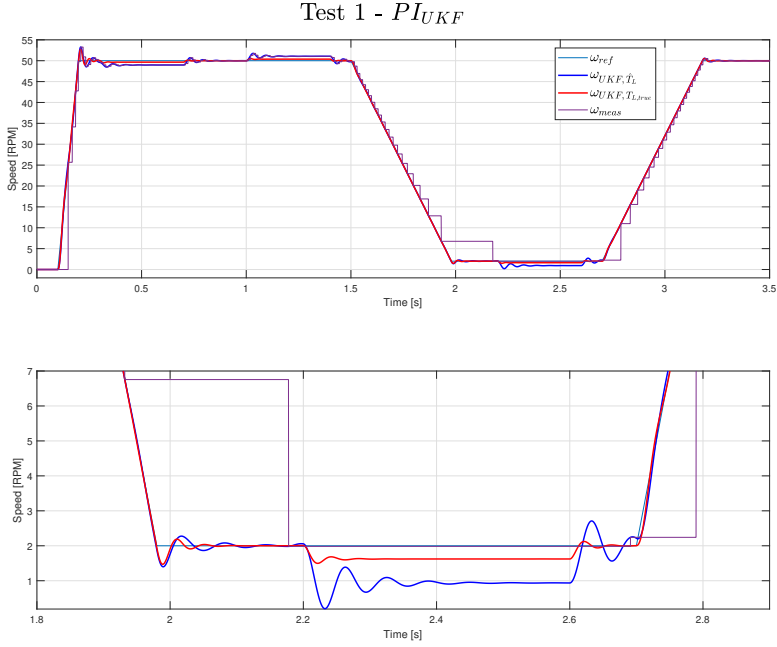


Figure 6.4: *PI-controlled motor with 100 ppr decoder resolution.*

In Figure 6.4 test 1 can be seen performed by a PI-controller and UKF along with a lowered encoder resolution from 4096 to 100 pulses per revolution, the speed measurement update frequency is slow at low RPMs but with the help of an UKF it is possible to control the motor with a resolution of 100 PPR ($\omega_{UKF, \hat{T}_L}$), which was not possible with 4096 PPR without a filter. Note that with an optimal torque load estimator the results ($\omega_{UKF, T_{L,true}}$) could be even better. For this case the UKF has been tuned to trust more in the model since the measurement update is less frequent, i.e., a smaller value of the Q-matrix .

6.2 Test 2 - Smooth start and stop

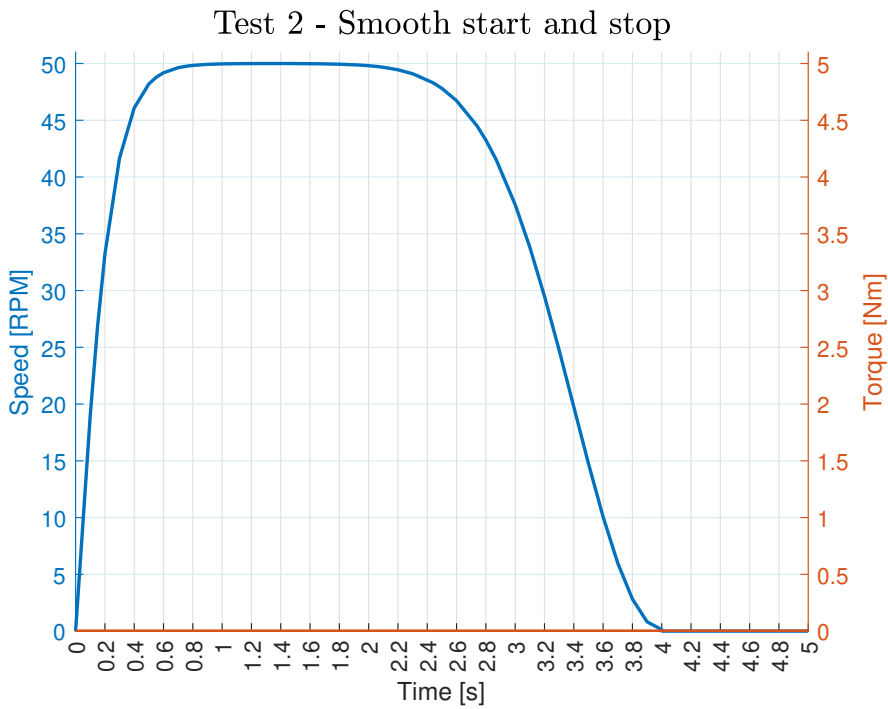


Figure 6.5: Reference speed for test 2.

Test 2, is shown in Figure 6.5. It is based on the expression "smooth start and stops" and it is assumed there is no load torque in this test. At 0 s the reference speed is hyperbolical increased until it reaches 50 RPM, to mimic a smooth start. At 1.6s it starts to exponential decrease in order to simulate a smooth stop.

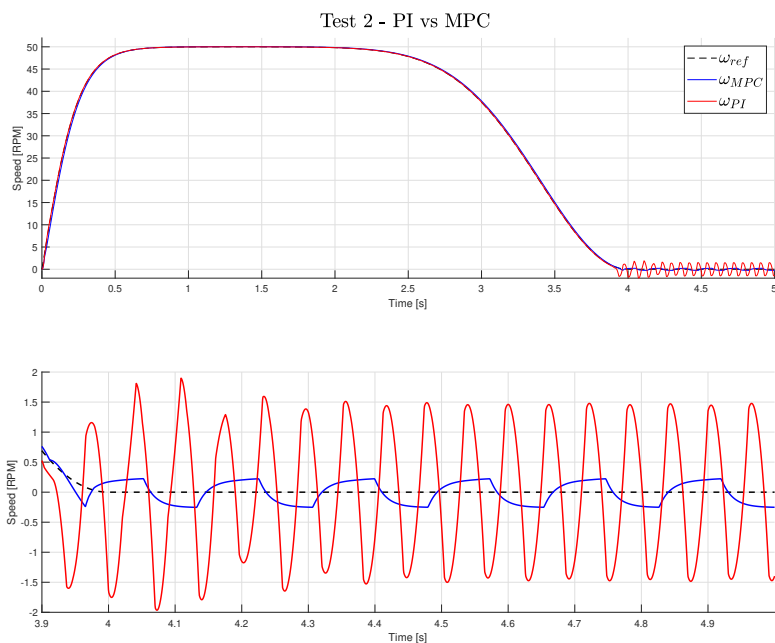


Figure 6.6: MPC and PI-controlled motor.

The results from test 2, Figure 6.6, shows that the the motor can easily be controlled at smooth operations. When the speed is approaching 0 RPM it begins to oscillate, which is displayed in the bottom subfigure, with both MPC and PI but milder oscillations with the MPC.

6.3 Test 3 - Speed torque

Test 3 is made to show the capability of controlling different stationary speeds at increasing torque load.

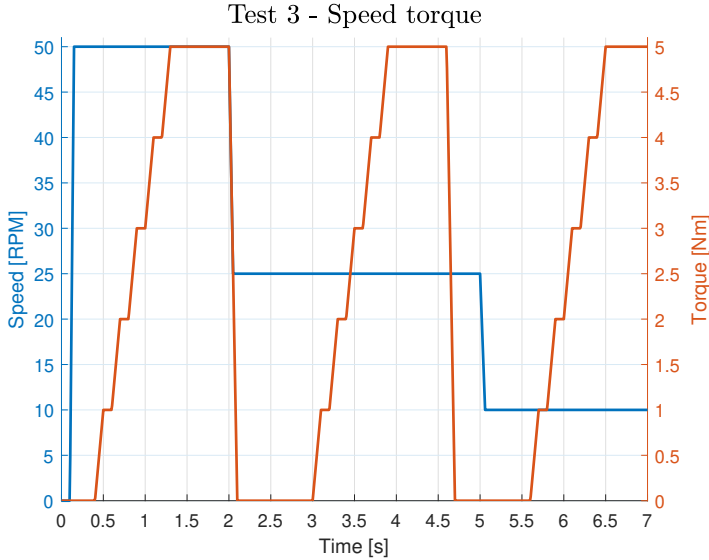


Figure 6.7: Reference speed and applied load torque for test 3.

Test 3, is shown in Figure 6.7. It is called speed torque for the reason that it sets the speed to three different levels; 50 RPM at 0.1s, 25 RPM at 2s and 10 RPM at 5s. For each speed the load torque is increasing incrementally over a time period of 1.5 s until reaches 5 Nm.

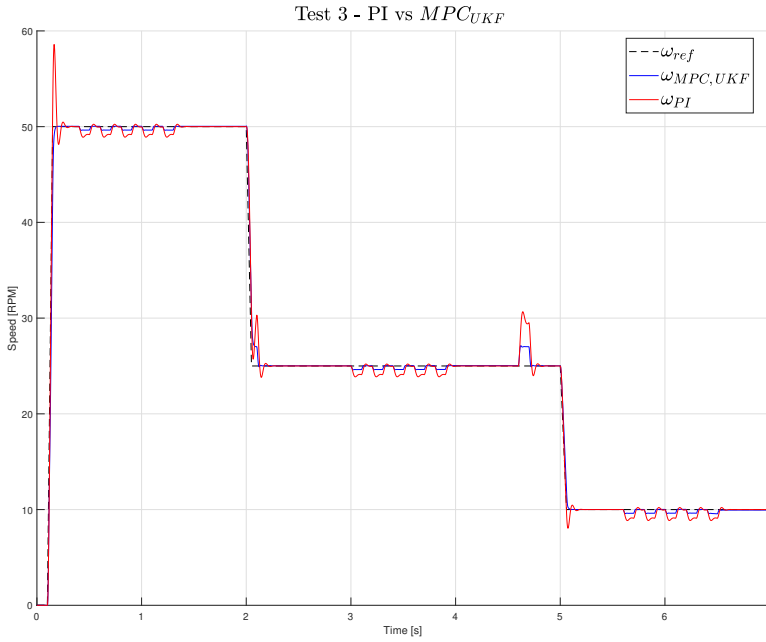


Figure 6.8: MPC controlled with UKF-estimated speed and PI controlled, with test 2 references.

The results from test 3, Figure 6.8, shows that a PI-controller can control the motor at the three different speeds with applied torque steps. At great acceleration and deceleration it gives a very clear over- and undershoot. The MPC is reducing the overshoots, at the initial one where the speed is accelerated from 0 to 50 RPM it is be reduced from almost 20% to zero.

6.4 Test 4 - Worst case

Test 4 is used to evaluate the control strategies at the highest torque load for the three specified speeds.

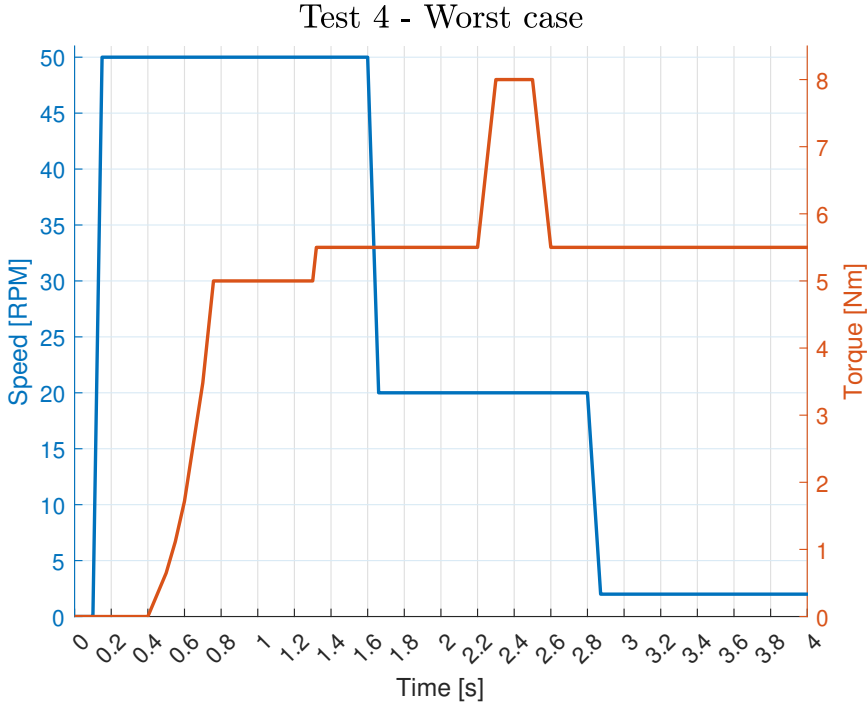


Figure 6.9: Reference speed and applied load torque for test 4.

Test 4, is shown in Figure 6.9. The reference speed is a ramp up to 50 RPM, then at 1.6s it is sloped down to 20 RPM and lastly at 2.8s down to 2 RPM. The load torque applied is at 0.4s a positive exponential curve up to 5 Nm, thereafter a ramp to 5.5 Nm. At worst case it is, at 2.2s, a ramp up to 8 Nm for 0.2s and then down to 5.5 Nm again.

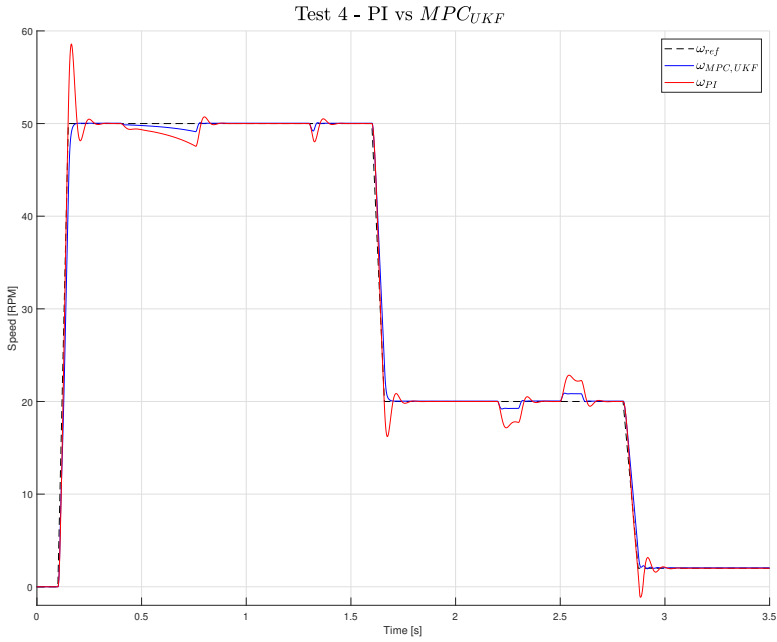


Figure 6.10: MPC controlled with UKF-estimated speed and PI controlled, with test 4 references.

For the worst case test, see Figure 6.10, both controllers manage to control the speed to the wanted reference as long as the torque is constant and the speed is greater than 2 RPM. The PI-controller gives heavy over- and undershoots at acceleration and deceleration while the MPC none. At constant reference speed, e.g. at 50 RPM, the speed decreases when torque load is applied and around 0.5s one can see that for the MPC configuration the speed drops less than for the PI. Lastly at 2 RPM the PI does not manage to prevent the motor from reversing.

7

Discussion

In this chapter the results and chosen methodology is discussed.

7.1 Modelling

In our model any non-linearities throughout one revolution that may be affected by the permanent magnets and their number & placement has not been considered. The phenomenon of torque ripples that has not occurred in the simulated model could have to do with these simplifications. This could also explain the non-parabolic shape the simulated speed-torque curve has, the reason for this given by the corporation is that the physical motor uses more powerful neodymium magnets which are not considered in the simulation.

When validating the model some parameters were changed, firstly viscous damping was set to $0.008 \frac{Nm \cdot s}{rad}$. The value is reasonable, in the same magnitude, looking at other reports of similar topic, and it is also reasonable to believe that some viscous damping exists even though it was not given in the data sheets. This parameter can be identified with help of physical tests, instead of the trial and error tuning used.

The magnetic flux linkage is adjusted from 0.09 to 0.1 Weber, it is a parameter that is hard to estimate and has on the same physical motor been estimated to many different values. It should also be noted that it is one of the parameters that affect the motor's behaviour the most, where a high magnetic flux makes it strong on low speeds with high torque load, but lowers the no-load top speed due to greater back-emf. Lowering the parameter has more or less the opposite effect, but this is not completely linear.

The phase resistance was changed from 1.633 to 3.19 Ω which is almost twice as big, this is notable since it is easy to measure this parameter, but another data-sheet for the same motor has given the phase resistance 3.19 Ω so there is an uncertainty on the given information and 3.19 Ω provides an improved model resemblance.

To improve the resemblance between the simulated model and the physical motor some parameter identification methodology could be performed, for example a grey-box model [14].

Validating the open loop model the simulated efficiency is about 8% greater than physical model, this can be due to the physical motor and its setup having many losses that are not seen for the simulated motor, such as rotor losses and hysteresis. Furthermore it could be noted that the physical motor has an overall low efficiency compared to what could be expected by an electrical machine. The answer to this, given by the corporation is because we are outside what would be an optimal power in this motor. This motor has almost only copper losses because of the low speed.

$$\eta = \frac{P_{in}}{P_{out}} = \frac{(P_{out} + P_{Losses})}{P_{out}} \text{ and } P_{out} = T_L \cdot \omega_m.$$

Since the torque is greater, we have greater copper losses and not that much power output.

In test 2, see Section 6.2, the controller has problems regulating the speed to zero. The reason for this is most likely the decoder making it impossible to reach zero speed. One should consider some separate solution for the back and forth rocking phenomenon, where one alternative could be locking the magnetic field when the speed is low enough and by this completely stopping the motor.

When modeling the filters, the measurement covariance matrix is set to $\mathbf{I}^{n \times n}$, this due to it having ideal and noiseless measurement signals in the simulated model giving the model no variance or covariance, this would have to be reconsidered in the case of implementation in a real model where one can calibrate and calculate the covariances.

Furthermore it can be noted that the measurement update of speed and angle is lower than the sampling speed of the filters, due to this the filters uses the same speed and angle measurements until it is updated. This knowledge could be used to tune the filter in a way such that it trusts the measurements more right when they are updated and then less when they are not.

Due to the UKF filter being based on the same mathematical model as the motor that is modeled, its prediction will come very close to the model output, with only the distorted speed and angle measurements along with the estimated torque load being different. In an implementation it would be impossible to make a perfect model and therefore the UKF could show lower performance and would

need to be tuned differently to what it is in the simulations.

Assuming the battery has unlimited power does not limit the amount of current that can be drawn from the motor and therefore neither the electrical torque produced by the motor. The motor draws around 4-5 ampere when a load torque of 5 Nm is applied, which is around a third of the hypothetical limit of 15 ampere that was given by the company. This way current restriction does not affect this work.

The assumption of an ideal inverter impacts the model such that the losses from it are neglected e.g. switching-losses when the electric parts are producing a three-phase voltage.

To limit the work to one motor instead of two mainly affects the modelling part of the thesis. An extra loop would be needed, controlling both motors and making them cooperate. On the other hand, a model with two motors would have resulting in a more realistic setup.

7.2 Controlling

At high acceleration and deceleration's the PI-controller is resulting in a large overshoot/undershoot, this phenomena can be prevented by adding the derivative part in the PI, making it a PID controller. This would perhaps work in the ideal system but when the signal is noisy a derivative part can cause problems in terms of magnifying the noise, which is a more realistic scenario of the physical system and that is mainly why it is not included in the thesis.

The model predictive control is predicting the error for a number of time steps and is calculating the optimal control input in order minimize the error to reach the estimated reference point. The result, in terms of MPC being smoother and better than the PI was the predicted outcome. However, the implementation of a MPC and the tuning of it is more advanced and non-intuitive than for a PI controller and the online optimization is more computationally heavy than for a PI controller. This may not have been taken into account in this thesis but could be worth to bare in mind for future implementation.

8

Conclusions

The conclusions that answers the problem statement will be presented as well as potential future work that could be considered for further development.

8.1 Conclusions

As seen in Figure 3.5 and 3.7 the simulation modeled in simulink does mimic the physical motor in terms of performance, the speed-torque curve follows each other closely, the difference being that the physical motor has a slightly parabolic shape. The voltages are similar, this due to the voltage being an input to the system. The current is initially and at the final value close to the physical model, but in between it's lower, this also affects the efficiency such that it's greater for the modeled values than for the physical ones. But overall, the results are such as they can be used to evaluate different control strategies on the model.

As seen in the results the PI can control the motor in most cases but at acceleration/deceleration it results in over-/undershoot and at 2 RPM it does not manage to keep the motor running forward without the UKF. When torque is continuously increased the PI-controller is resulting in the speed dropping. The MPC controls the motor better with no overshoot and less stationary error when the torque is continuously increased. Finally, none of the controllers is able to make the motor stop smoothly.

Both the EKF and UKF estimation is closer to the true speed at lower RPMs, when measurement update is slow, than the position sensor. Thereby it contributes with an increase of the controller's performance. In test 1 the PI-controller could not control the motor without it reversing using an encoder resolution of 4096 PPR, while with an UKF the controller did manage to keep the speed positive.

8.2 Future work

Both the modelling and controlling of the motor could be improved, most foremost the modelling of the motor since it is assumed to be ideal. The things that may be further investigated and developed are listed below:

- Losses as windage-losses, rotor losses (induced eddy currents) and stator losses (copper and iron losses) can be modelled for a more accurate model of the PMSM. The type of permanent magnet can also be taken into consideration as well as the inverter, for an even more realistic model.
- Parameter estimation of the physical motor can be identified with more reliable tests.
- To get a fair physical comparison of the different controllers they can be implemented on a microcontroller and thereafter tested with the physical motor in order to investigate the required computing time and computing power.
- Validation of the model can be made with different motors to ensure model robustness against individual variations.

Appendix

A

Appendix

A.1 Motor requirements

Motor operations		Conditions	Period [min]	Duty Cycle [%]	Shaft Torque [Nm]	Shaft speed [RPM]
Operating mode	worst case	max slope, max acceleration at near max speed	2	20	5	50
		Stuck at max slope, wheel imbalanced.	0.5	2	8	20
Docking	initial docking at slope: acceleration mode.	7-deg slope max acceleration at docking speed.	1	10	2	4
	final docking at slope: acceleration mode.	7-deg slope max acceleration at half the docking speed with 50 Nm final push.	0.3	5	5.5	2
Searching	slope ground: random search, acceleration mode	max slope, max acceleration, at near max speed	5	10	4.5	50
	slope ground: random search, constant velocity mode	max slope, zero acceleration, at near max speed	20	30	4	50
	slope ground: following guide, acceleration mode	max slope, max acceleration, at half speed	3	20	4.5	25

Table A.1: Requirements that the controller is to conduct.

A.2 Controller parameters

In this appendix the parameters used for the controller setup is presented in four different tables listed below. Except the information that is in the tables the observer gain was set to $K = [1796 \ -4275]^\top$

PI-settings			
	ω_m - controller	i_d - controller	i_q - controller
K_p [-]	0.5	35	35
K_i [-]	50	5000	5000
Freq [Hz]	10000	10000	10000

Table A.2: Parameters used for the PI-controller only.

PI - Settings				EKF-settings		UKF-settings	
	ω_m - loop	i_d - loop	i_q - loop	Q [-]	diag([1000 1000 10 10])	Q [-]	diag([1000 1000 10 10])
K_p [-]	0.5	35	35	R [-]	$\mathbb{I}^{4 \times 4}$	R [-]	$\mathbb{I}^{4 \times 4}$
K_i [-]	100	5000	5000	T_s [s]	0.0001	T_s [s]	0.0001
Freq [Hz]	10000			v [-]	$\sim \mathcal{N}(0, 0)$	v [-]	$\sim \mathcal{N}(0, 0)$

Table A.3: Parameters used for the PI-controller with the two Kalman filters.

MPC-settings		Min & max		
N [-]	50	ω_m [RPM]	0	50
h [s]	0.001	T_L [Nm]	-8.5	8.5
Q [-]	diag([1 800])	i_q [A]	-30	30
R [-]	4000	v_q [V]	-20	20
Q_N [-]	diag([1 50])	-	-	-

Table A.4: Parameters used for the MPC only.

MPC-settings		Min & max		EKF-settings		UKF-settings	
N [-]	50	ω_m [RPM]	0 50	Q [-]	diag([1000 1000 10 10])	Q [-]	diag([1000 1000 10 10])
h [s]	0.001	T_L [Nm]	-8.5 8.5	R [-]	$\mathbb{I}^{4 \times 4}$	R [-]	$\mathbb{I}^{4 \times 4}$
Q [-]	diag([1 800])	i_q [A]	-30 30	T_s [s]	0.0001	T_s [s]	0.0001
R [-]	5000	v_q [V]	-20 20	v_k [-]	$\sim \mathcal{N}(0, 0)$	v_k [-]	$\sim \mathcal{N}(0, 0)$
Q_N [-]	diag([1 5])	-	-	-	-	-	-

Table A.5: Parameters used for the MPC with the two Kalman filters.

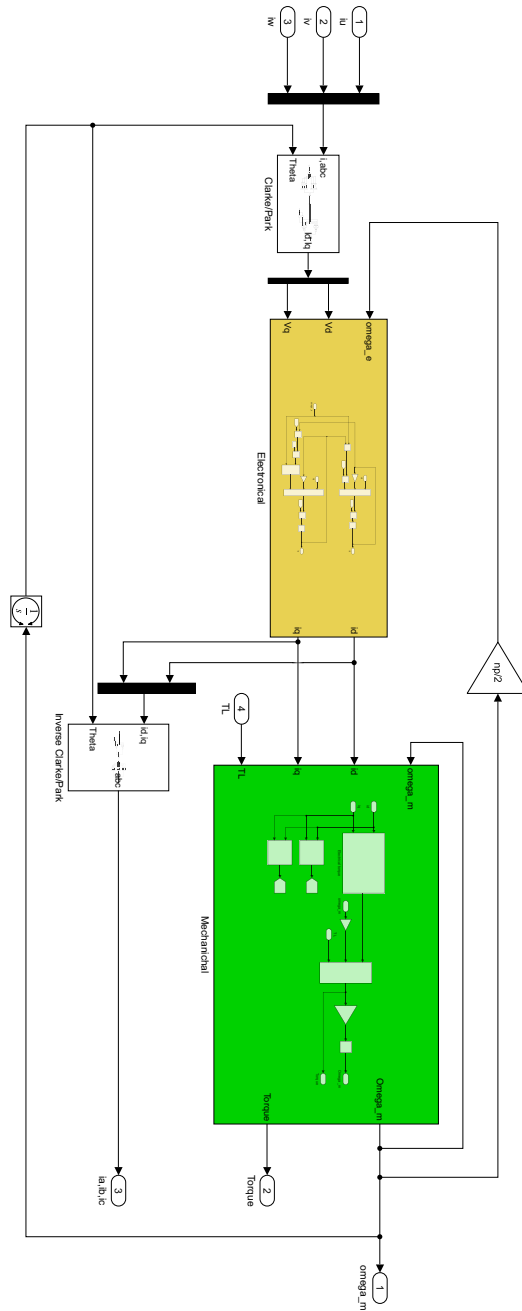


Figure A.4: Simulink model of the PMSM.

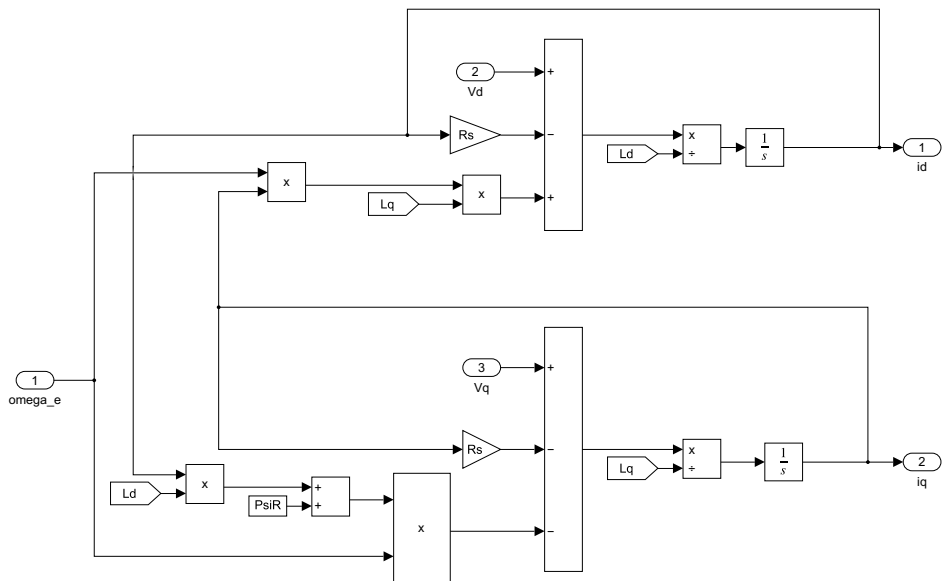


Figure A.5: Simulink model of the electrical part of the PMSM.

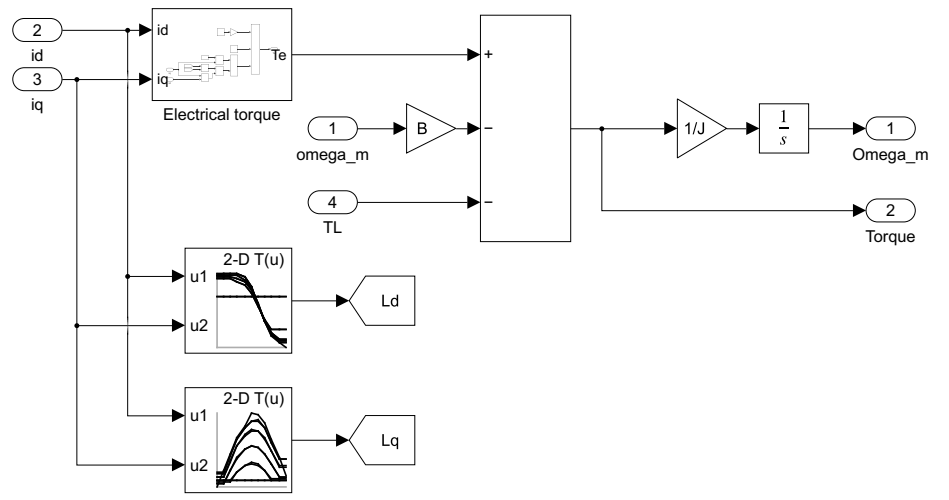


Figure A.6: Simulink model of the mechanical part of the PMSM.

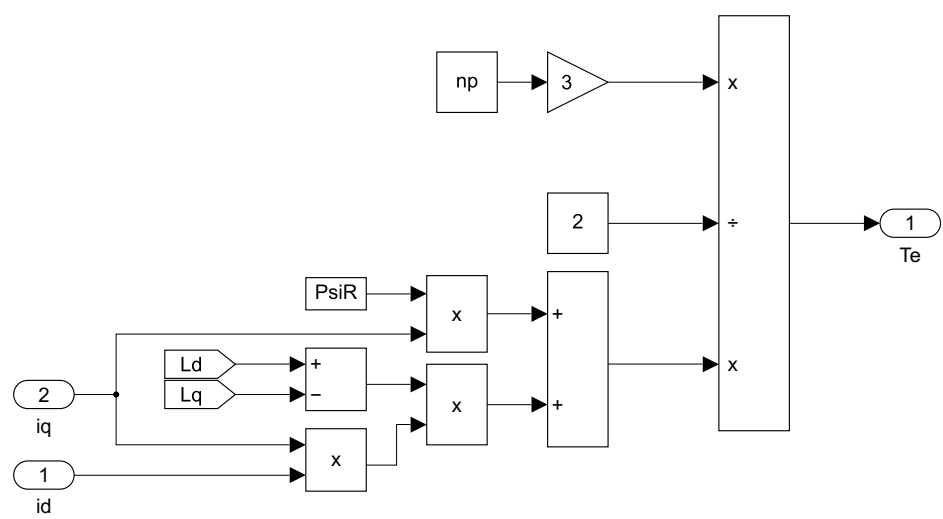


Figure A.7: Simulink model of the electrical torque-part of the PMSM.

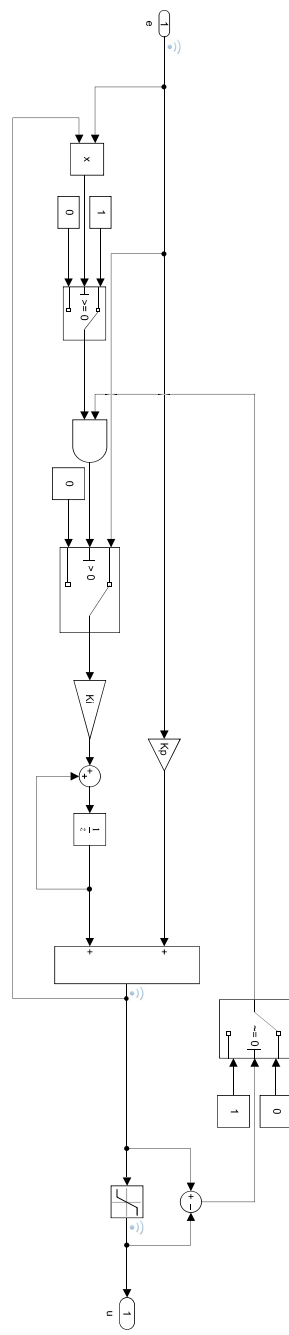


Figure A.8: Simulink model of anti-windup.

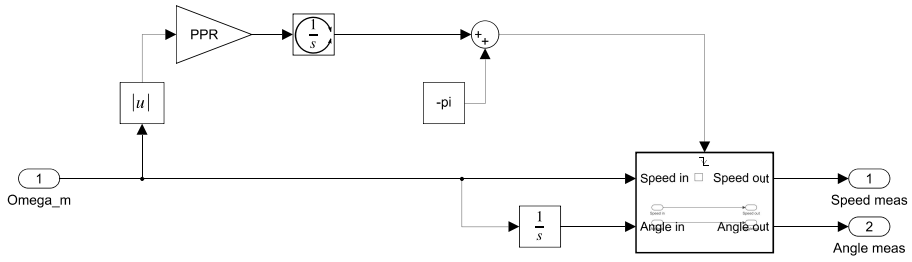


Figure A.9: Simulink model of the implemented decoder.

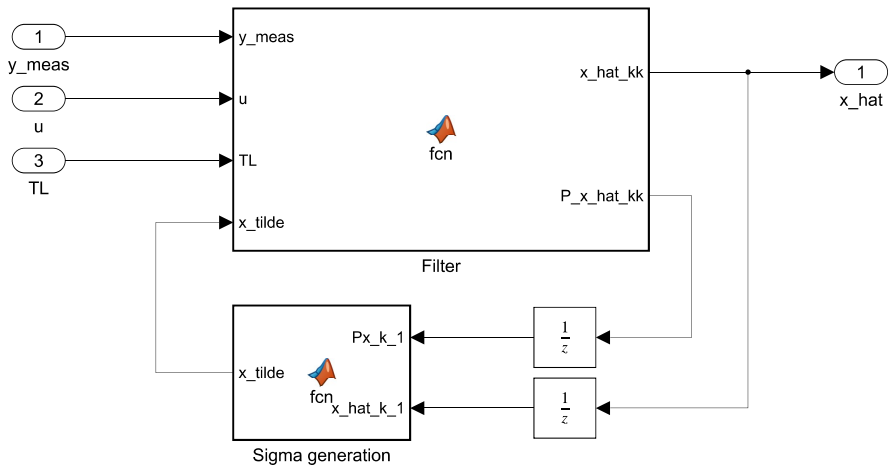


Figure A.10: Simulink model of the Unscented Kalman Filter.

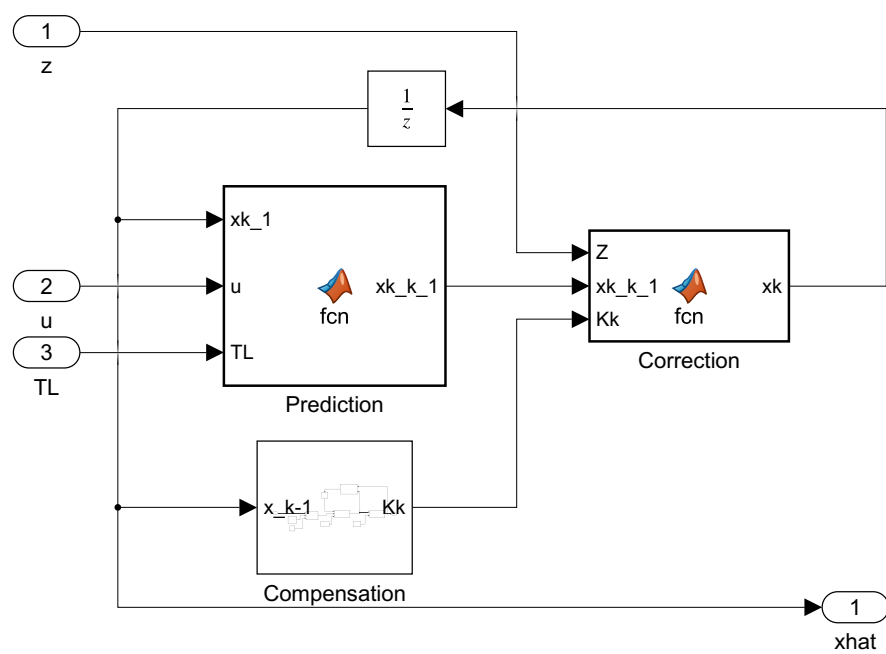


Figure A.11: Simulink model of the Extended Kalman Filter.

A.4 EKF/UKF algorithm

```

1 function x_tilde = fcn(Px_k_1, x_hat_k_1)
2     n = 4; % Number of states
3     x_tilde = zeros(4,9);
4
5     x_tilde(:,1) = x_hat_k_1; % First Sigma point
6     chol_matrix = chol(n*Px_k_1);
7     for i = 1:n
8         x_tilde(:,1+i) = x_hat_k_1 + chol_matrix(i,:)' ; % Four positive
          sigma points
9         x_tilde(:,n+1+i) = x_hat_k_1 - chol_matrix(i,:)' ; % Four Negative
          sigma points
10    end
11 end

```

Listing A.1: UKF Sigma generation

```

1 function [x_hat_kk, P_x_hat_kk] = fcn(y_meas, u, TL, x_tilde)
2
3     % Constants given from outside the UKF
4     np = var(1);
5     Ld = var(2);
6     Lq = var(3);
7     PsiR = var(4);
8     Rs = var(5);
9     J = var(6);
10    b = var(7);
11
12    % Input to the system
13    ud_meas = u(1);
14    uq_meas = u(2);
15
16    n = 4; % Number of states
17
18    Q = diag([1 1 1 1]); % Measurement noise
19    R = diag([1 1 1 1]); % Process noise
20
21    dt = 1e-4; % Sampling frequency
22
23    UTout = zeros(4,9);
24    for i = 1:length(x_tilde(1,:))
25        id_hat = x_tilde(1,i);
26        iq_hat = x_tilde(2,i);
27        wm_hat = x_tilde(3,i);
28        theta_hat = x_tilde(4,i);
29
30        UTout(1,i) = -Rs/Ld*id_hat + wm_hat*np*Lq/Ld*iq_hat + ud_meas/Ld;
31        UTout(2,i) = -Rs/Lq*iq_hat - wm_hat*np*Ld/Lq*id_hat + uq_meas/Lq -
          wm_hat*np*PsiR/Lq;
32        UTout(3,i) = (3*np/2*(PsiR*iq_hat + (Ld-Lq)*id_hat*iq_hat)-b*
          wm_hat-TL)/J;
33        UTout(4,i) = wm_hat;
34    end
35
36    x_hat_kk_1_i = zeros(4,9);
37    for i = 1:length(x_tilde(1,:))

```

```

38     x_hat_kk_1_i(:,i) = x_tilde(:,1) + UTout(:,i)*dt; % Transformed
        sigma points
39     end
40
41     x_hat_kk_1 = zeros(4,1);
42     for i = 1:n
43         x_hat_kk_1(i,1) = 1/length(x_hat_kk_1_i(1,:))*sum(x_hat_kk_1_i(i
        ,:)); % State mean
44     end
45
46
47     P_x_hat_kk_1 = zeros(4);
48     for i = 1:length(x_hat_kk_1_i(1,:))
49         P_x_hat_kk_1_calc = (x_hat_kk_1_i(:,i) - x_hat_kk_1) * (
        x_hat_kk_1_i(:,i) - x_hat_kk_1)';
50         P_x_hat_kk_1 = P_x_hat_kk_1 + P_x_hat_kk_1_calc;
51     end
52     P_x_hat_kk_1 = 1/length(x_hat_kk_1_i(1,:))*P_x_hat_kk_1 + Q; % State
        covariance
53
54     y_hat_i = x_hat_kk_1_i;
55
56     y_hat = zeros(4,1);
57     for i = 1:n
58         y_hat(i,1) = 1/length(x_hat_kk_1_i(1,:))*sum(y_hat_i(i,:)); %
        Measurement mean
59     end
60
61
62     P_y_hat = zeros(4);
63     for i = 1:length(x_hat_kk_1_i(1,:))
64         P_y_hat_calc = (x_hat_kk_1_i(:,i) - x_hat_kk_1) * (y_hat_i(:,i
        ) - y_hat)';
65         P_y_hat = P_y_hat + P_y_hat_calc;
66     end
67     P_y_hat = 1/length(x_hat_kk_1_i(1,:))*P_y_hat + R; % Measurement
        covariance
68
69     P_xy_hat = zeros(4);
70     for i = 1:length(x_hat_kk_1_i(1,:))
71         P_xy_hat_calc = (x_hat_kk_1_i(:,i) - x_hat_kk_1) * (y_hat_i(:,
        i) - y_hat)';
72         P_xy_hat = P_xy_hat + P_xy_hat_calc;
73     end
74     P_xy_hat = 1/length(x_hat_kk_1_i(1,:))*P_xy_hat; % State and
        measurement covariance
75
76     K = P_xy_hat*(P_y_hat^-1); % Kalman gain
77
78     x_hat_kk = x_hat_kk_1 + K*(y_meas - x_hat_kk_1); % State estimate
79     P_x_hat_kk = P_x_hat_kk_1 - K*P_y_hat*K'; % state covariance
80 end

```

Listing A.2: UKF Filter

```

1 function xk_k_1 = fcn(xk_1,u,TL)
2
3
4
5 x1 = xk_1(1); %id
6 x2 = xk_1(2); %iq
7 x3 = xk_1(3); %omega
8 x4 = xk_1(4); %theta
9
10 %Input to the system
11 vd = u(1);
12 vq = u(2);
13
14 % Constants given from outside the EKF.
15 np = var(1);
16 Ld = var(2);
17 Lq = var(3);
18 PsiR = var(4);
19 Rs = var(5);
20 J = var(6);
21 b = var(7);
22
23 dt = 1e-4; % Sampling frequency
24
25
26 f = [ -Rs/Ld*x1 + x3*Lq/Ld*x2*np + 1/Ld*vd ;
27       -Rs/Lq*x2 - x3*Ld/Lq*x1*np + 1/Lq*vq - x3*PsiR/Lq*np;
28       (3*np/2*(PsiR*x2 + (Ld-Lq)*x1*x2) - b*x3 - TL)/J;
29       x3
30       ];
31
32 %forward euler
33 xk_k_1 = xk_1 + dt*f;
34 end

```

Listing A.3: EKF Prediction

```

1 function xk = fcn(Z,xk_k_1,Kk)
2
3 H = eye(4);
4
5 xk = xk_k_1 + Kk*(Z - H*xk_k_1);
6 end

```

Listing A.4: EKF Correction

```

1 function Jk = jac(x)
2
3
4
5 x1 = x(1); %id
6 x2 = x(2); %iq
7 x3 = x(3); %omega
8 x4 = x(4); %theta
9
10 % Constants given from outside the EKF.
11 np = var(1);
12 Ld = var(2);

```

```

13 Lq = var(3);
14 PsiR = var(4);
15 Rs = var(5);
16 J = var(6);
17 b = var(7);
18
19 Q = diag([1 1 1 1]); % Measurement noise
20 R = diag([1 1 1 1]); % Process noise
21
22 dt = 1e-4; % Sampling frequency
23
24
25 Jk = [1-Rs*dt/Ld                                dT*Lq/Ld*x3*np
        dT*Lq/Ld*x2                                0;      %id
        -dT*Ld/Lq*x3*np                            1-Rs*dt/Lq
        -dT*(Ld/Lq*x1 + PsiR/Lq)                    0;      %iq
        3*np/2*(Ld-Lq)*x2*dT/J                      3*np/2*(PsiR + (Ld-Lq)*x1)*dT/J
        1-b/J*dT                                      0;      %omega
        0                                              0
        dT                                             1];    %theta
29
30 H = eye(4); %Measurment Jacobian
31
32 Pk_k_1 = Jk*P_k_1*Jk' + Q;
33
34 Kk = Pk_k_1*H'*inv(H*Pk_k_1*H'+R);
35
36 Pk = Pk_k_1 - Kk*H*Pk_k_1;
37
38 end

```

Listing A.5: EKF Kalman gain and covariance calculation.

Bibliography

- [1] Faisal Amin, Erwan Sulaiman, and Hassan Soomro. Field oriented control principles for synchronous motor. *International Journal of Mechanical Engineering and Robotics Research*, 8:284–288, 01 2019. doi: 10.18178/ijmerr.8.2.284-288.
- [2] Joel A E Andersson, Joris Gillis, Greg Horn, James B Rawlings, and Moritz Diehl. CasADi – A software framework for nonlinear optimization and optimal control. *Mathematical Programming Computation*, In Press, 2018.
- [3] S. Chattopadhyay, M. Mitra, and S. Sengupta. *Electric Power Quality*. Springer Dordrecht Heidelberg London New York, 2011. ISBN 978-94-007-0634-7. doi: 10.1007/978-94-007-0635-4.
- [4] Simulink Documentation. Simulation and model-based design, 2020. URL <https://www.mathworks.com/products/simulink.html>.
- [5] M. Enqvist, T. Glad, S. Gunnarsson, P. Lindskog, L. Ljung, J. Löfberg, T. McKelvey, A. Stenman, and J-E Strömberg. *Industriell reglerteknik, Kurskompendium*. Linköpings University, 2014.
- [6] T. Glad and L. Ljung. *Reglerteknik: Grundläggande teori*. Studentlitteratur AB, 2006. ISBN 978-44-02275-8.
- [7] Qian Guo and Tianhong Pan. Constrained explicit model predictive control with load state observer for pmsm drive system. In *2021 IEEE International Conference on Predictive Control of Electrical Drives and Power Electronics (PRECEDE)*, pages 291–295, 2021. doi: 10.1109/PRECEDE51386.2021.9680904.
- [8] D.C Hanselman. *Brushless Permanent-Magnet Motor Design*. R. R. Donnelley & Sons Company, 1994. ISBN 0-07-026025-7.
- [9] Tangqing Hu and Xuxiu Zhang. Simulation of pmsm vector control system based on fuzzy pi controller. In *2019 IEEE International Conference on Power, Intelligent Computing and Systems (ICPICS)*, pages 111–114, 2019. doi: 10.1109/ICPICS47731.2019.8942439.

- [10] Mohd Hafiz A. Jalil, Rohaiza Hamdan, Rafidah Ngadengon, Haslizamri Md Shariff, Siti Amely Jumaat, and Syed Zahurul Islam. Pid with clamping anti-windup performance on temperature regulation of glycerin bleaching process. In *2021 IEEE 12th Control and System Graduate Research Colloquium (ICSGRC)*, pages 127–131, 2021. doi: 10.1109/ICSGRC53186.2021.9515200.
- [11] S.J. Julier and J.K. Uhlmann. Unscented filtering and nonlinear estimation. *Proceedings of the IEEE*, 92(3):401–422, 2004. doi: 10.1109/JPROC.2003.823141.
- [12] Krishnamurthy P. Khorrami, F and H. Melkote. *Modelling and Adaptive Nonlinear Control of Electric Motors*. Springer, 2003. ISBN 3-540-00936-1.
- [13] David Lanlege, Rotimi Kehinde, Dolapo Sobanke, Abdulrganiyu Abdulrahman, and Umar Garba. Comparison of euler and range-kutta methods in solving ordinary differential equations of order two and four. pages 10–37, 06 2018.
- [14] Glad T. Ljung, L and A. Hansson. *Modelling and Identification of Dynamic Systems*. Studentlitteratur, 2021. ISBN 978-91-44-15345-2.
- [15] Dr .J McNames. Unscented kalman filter, ece 5/610 state space tracking, 2016.
- [16] Manfred Morari and Jay H. Lee. Model predictive control: past, present and future. *Computers Chemical Engineering*, 23(4): 667–682, 1999. ISSN 0098-1354. doi: [https://doi.org/10.1016/S0098-1354\(98\)00301-9](https://doi.org/10.1016/S0098-1354(98)00301-9). URL <https://www.sciencedirect.com/science/article/pii/S0098135498003019>.
- [17] Wanno Yeetum and Vijit Kinnares. Pi controller based on direct synthesis method for dc-link voltage control of active power filter. In *2018 International Electrical Engineering Congress (iEECON)*, pages 1–4, 2018. doi: 10.1109/IEECON.2018.8712250.
- [18] Muzy A. Zeigler, P and E. Kofman. *Theory of Modeling and Simulation Discrete Event and Iterative System Computational Foundations*. Academic Press, 2018. ISBN 978-0-12-813370-5.
- [19] Yifang Zhu, Gang Xu, Jian Yin, and Yuan Liu. Speed control of permanent magnet synchronous motor drives based on model predictive control. In *2018 IEEE 3rd International Conference on Cloud Computing and Internet of Things (CCIoT)*, pages 88–93, 2018. doi: 10.1109/CCIoT45285.2018.9032582.